

LESSON 23

Image classification
Intelligent vision System,
Object detection,
Segmentation and Recognition,
Machine Learning Hardware





Outline

- Image classification and Intelligent Vision Systems
 - Object detection, Segmentation, Recognition
 - Classical, Deep and Hybrid solutions
 - Use cases
- Machine learning Hardware
 - GPU vs CPU
 - TPU
 - Optimization tools: the OpenVINO example
 - General purpose Python based platform: Edge IMPULSE

Read the MATLAB short ebook on Deep Learning (not in the exam)

It resumes a lot of concepts discussed in the lesson and gives some code examples.





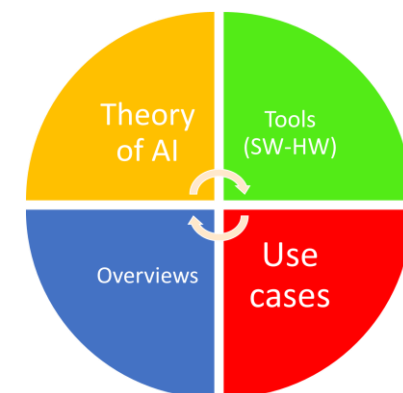
Dog



Cat

THEORY

Image classification



Intelligent Vision tasks

Semantic Segmentation



GRASS, CAT,
TREE, SKY

No objects, just pixels

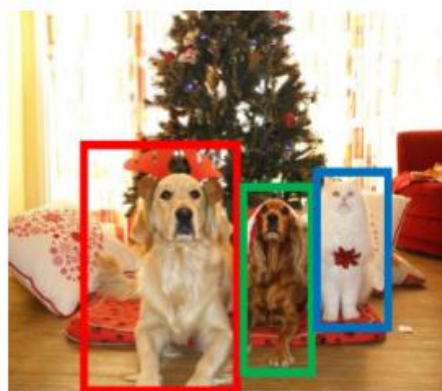
**Classification
+ Localization**



CAT

Single Object

**Object
Detection**



DOG, DOG, CAT

Multiple Object

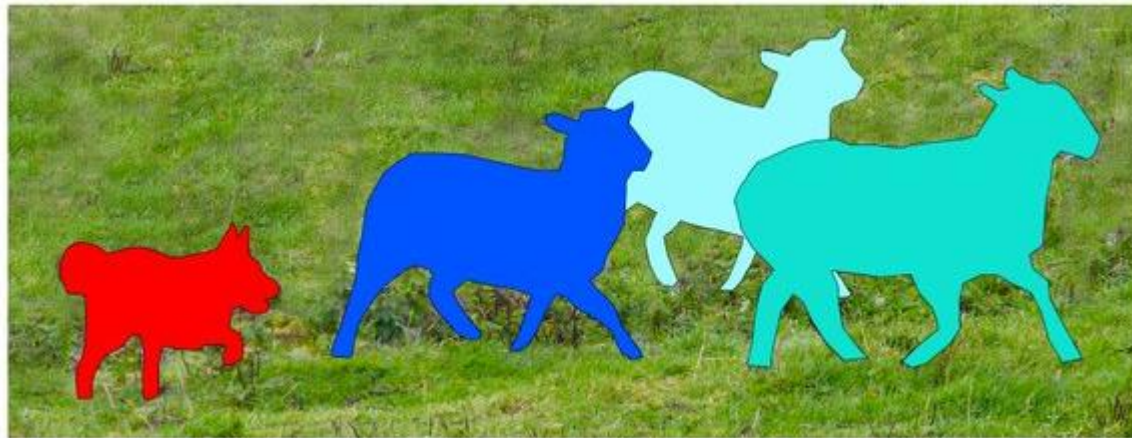
**Instance
Segmentation**



DOG, DOG, CAT

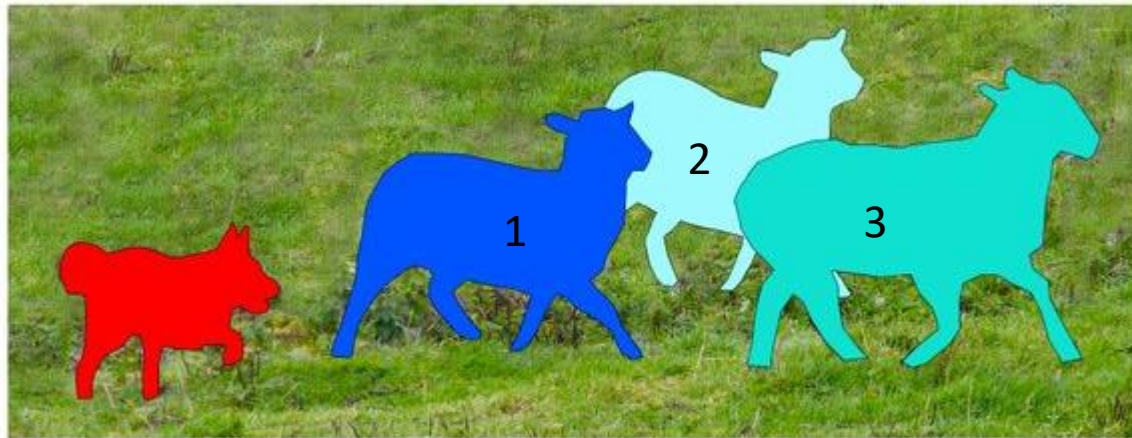
[This image is CC0 public domain](#)

A quick test 1 ...



What is this Intelligent vision task?

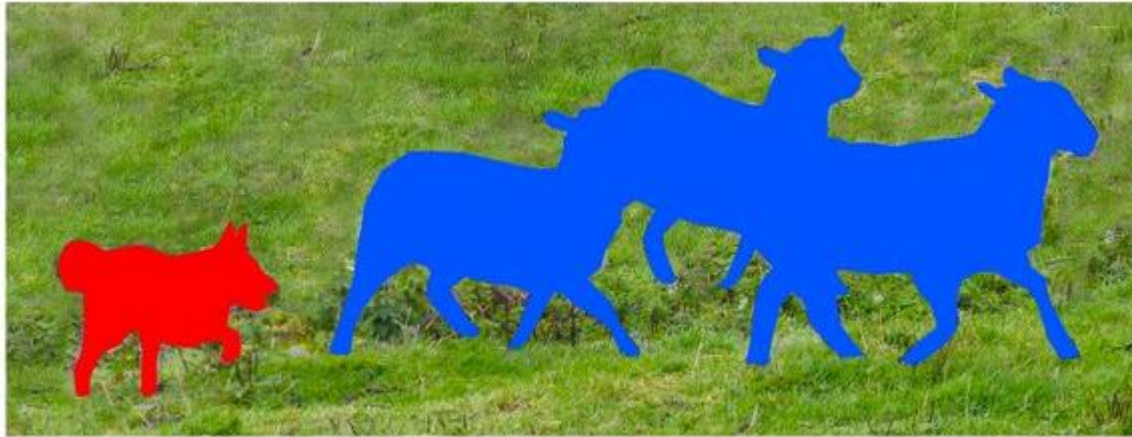
A quick test 1



What is this Intelligent vision task?

INSTANCE SEGMENTATION!

A quick test 2 ...



Which intelligent vision task does this correspond to?

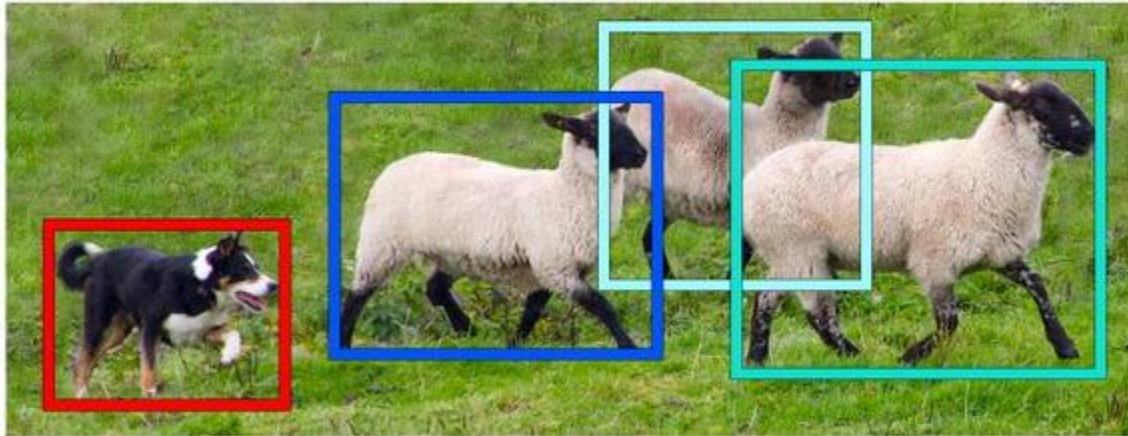
A quick test 2 ...



What is this Intelligent vision task?

SEMANTIC SEGMENTATION!

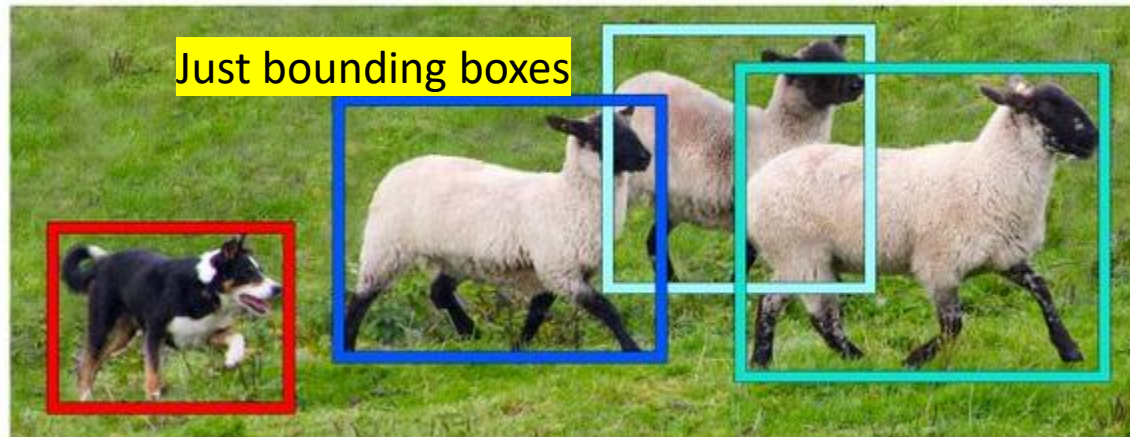
A quick test 3 ...



What is this Intelligent vision task?

A quick test 3 ...

We have just the so-called bounding box



What is this Intelligent vision task?

OBJECT DETECTION

A quick test 4 ...



What Intelligent vision task does it correspond?

A quick test 4 ...

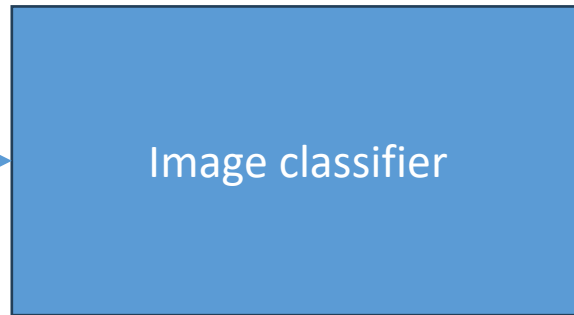
We have just one class in output = Sheep



What is this Intelligent vision task?

IMAGE CLASSIFICATION

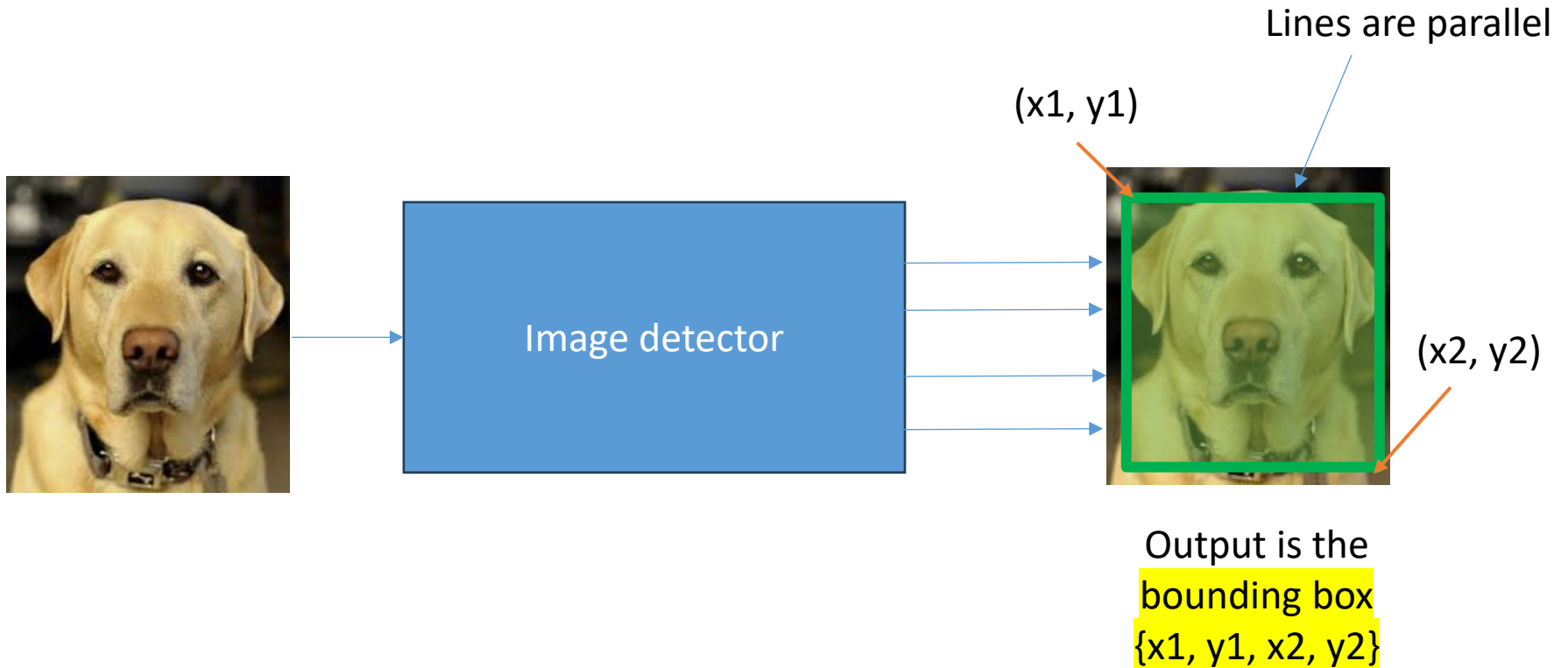
Direct image classification



«Dog»

No more
information
about the object

Object detection bounding box



It's almost impossible to get measurements

Direct image classification



Image classification

«Dog»

No more
information
about the object

Direct image classification: Example with AlexNet

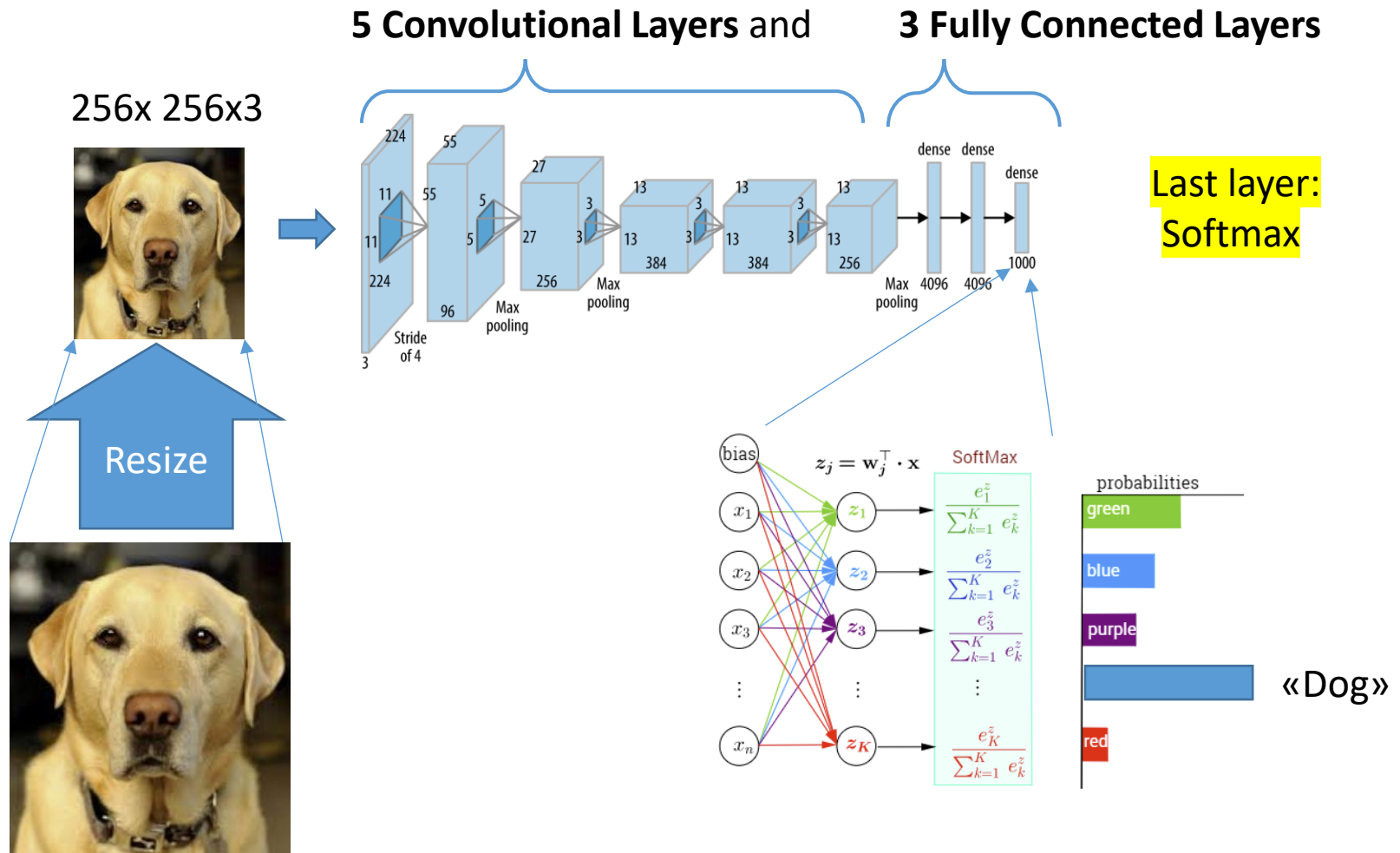


Image classification

Good for training
and fine tuning
(after selection)

EASY SET



HARD SET

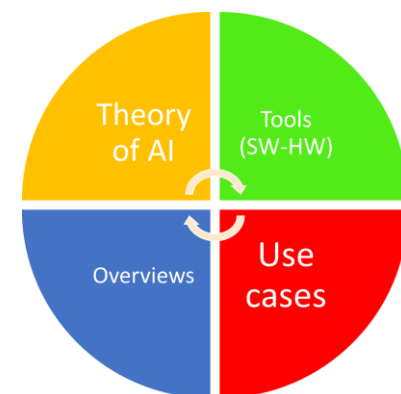
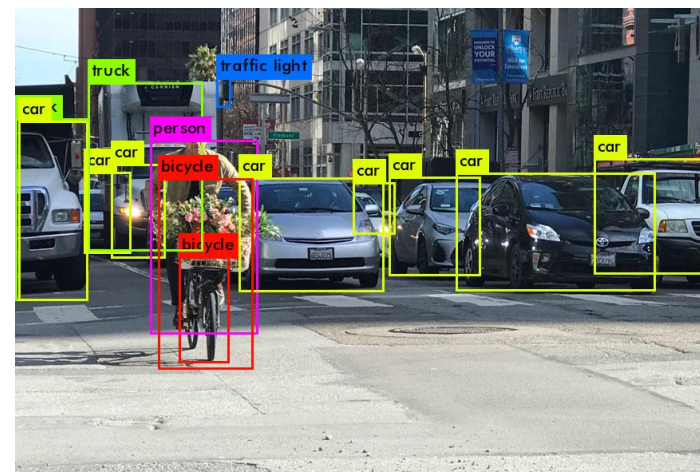


Good for Google...
not for an industrial
application...
But good to train to
extract feature!



THEORY

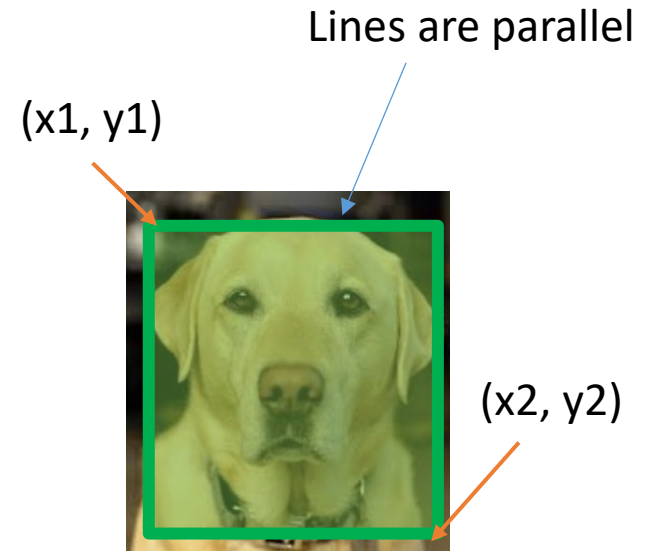
Object detection



Object detection bounding box



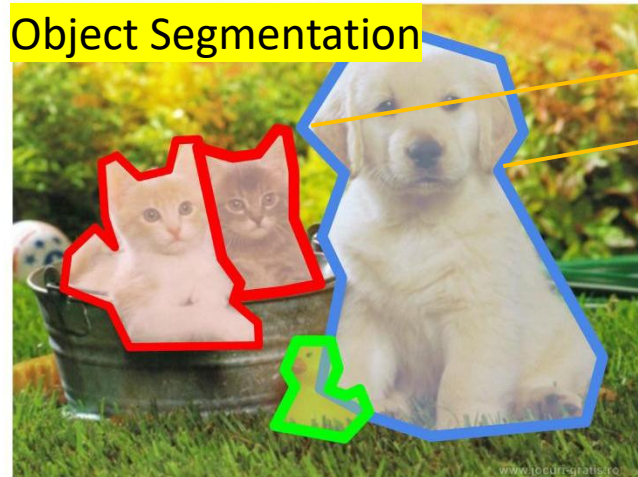
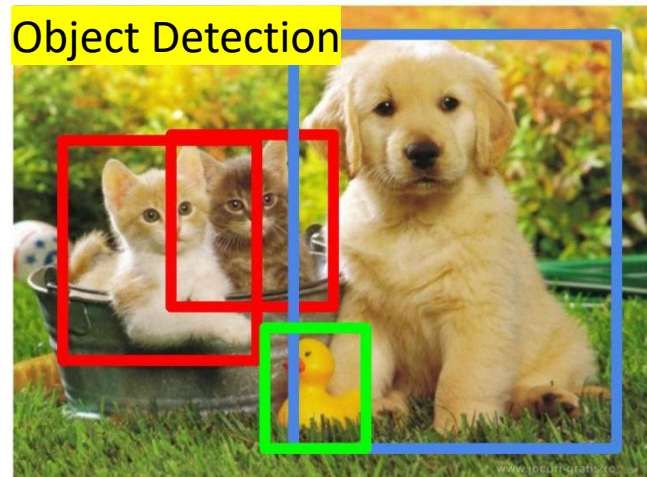
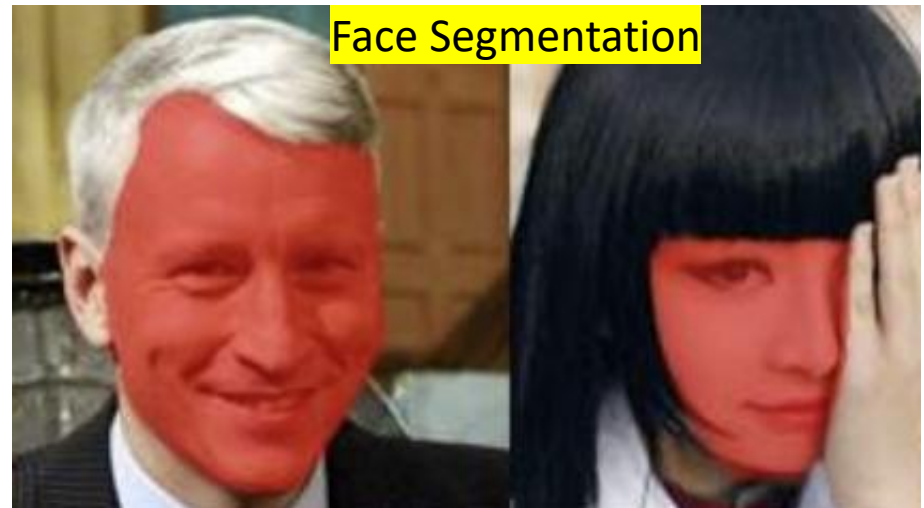
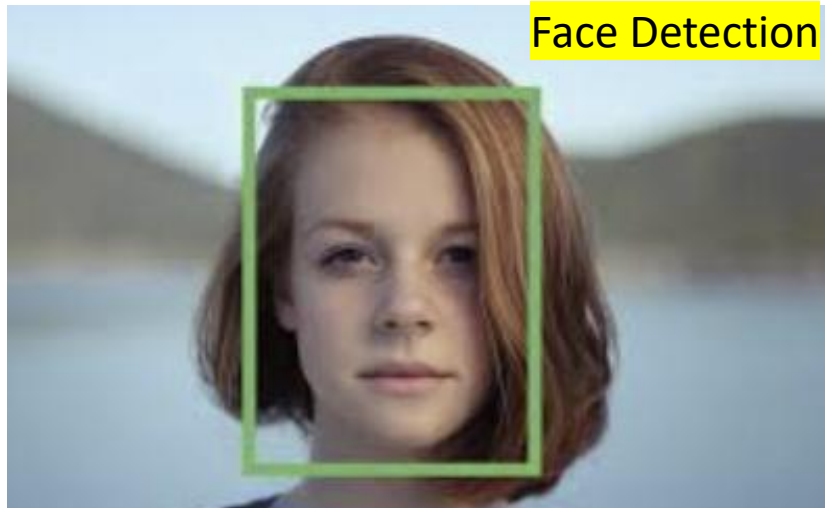
Object detection



Output is the
bounding box
 $\{x1, y1, x2, y2\}$

It's almost impossible to get measurements

Detection is not segmentation!

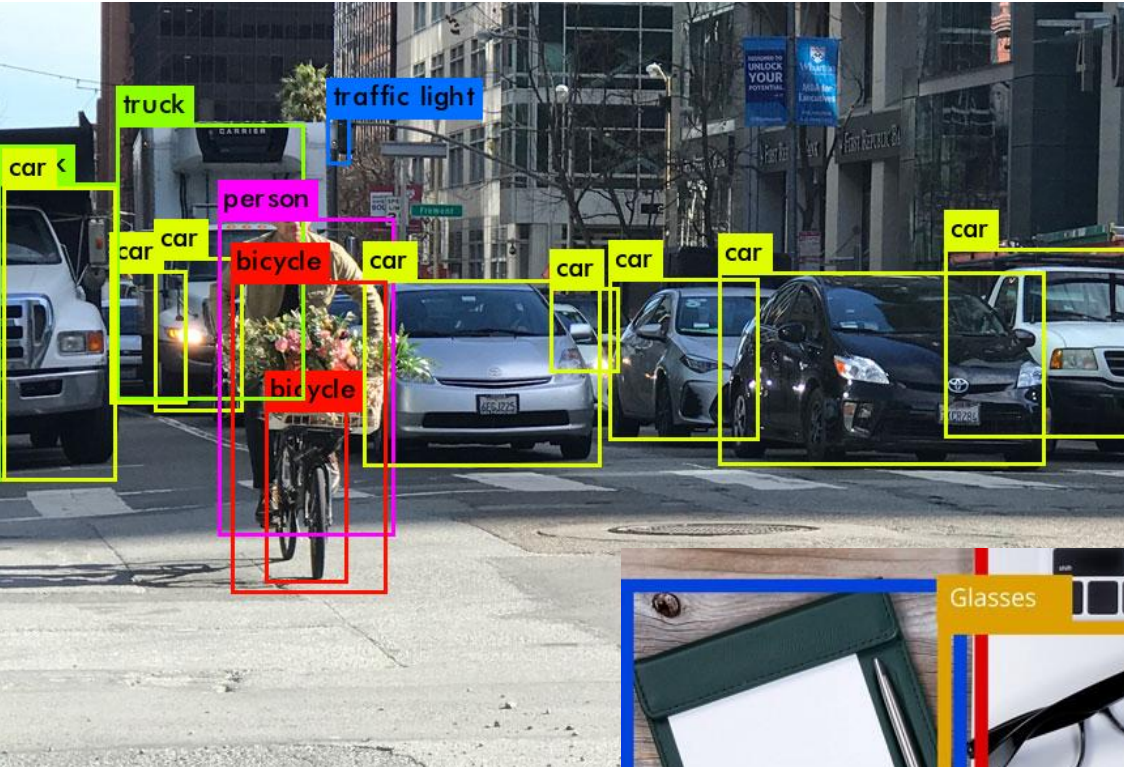


Real measures
of the objects!

Multiple object detection

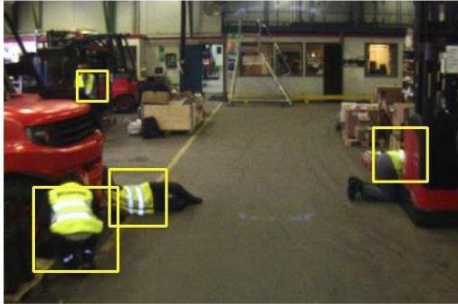
Not an easy task
in complex scenes

Overlapping objects



Detection

Example: person detection in images



Industrial sites



Surveillance



Articulated body pose estimation: OpenPose

OpenPose is an open-source real-time system for multi-person 2D pose estimation, including body, foot, wrist/hand, and facial keypoints

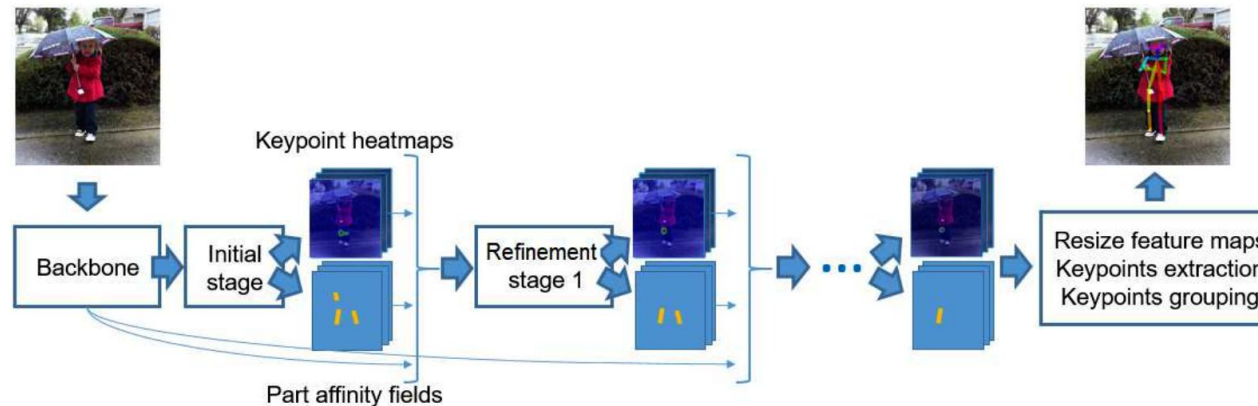
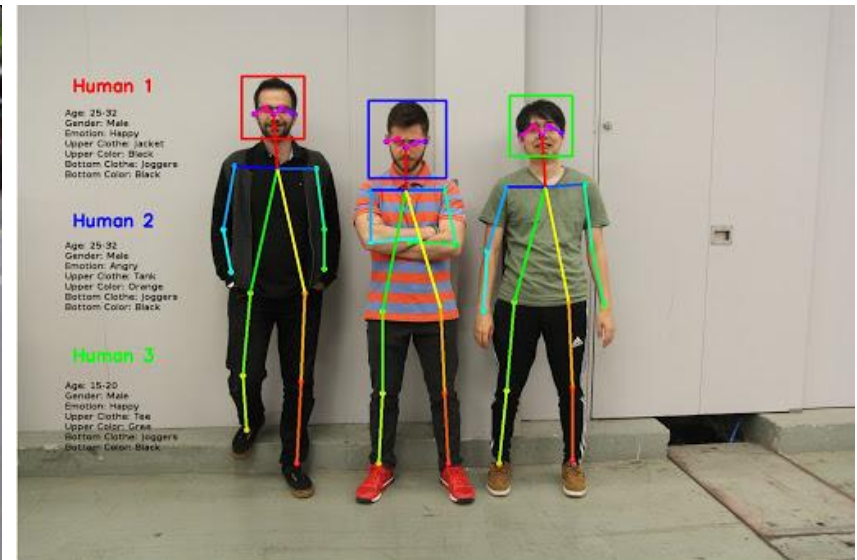
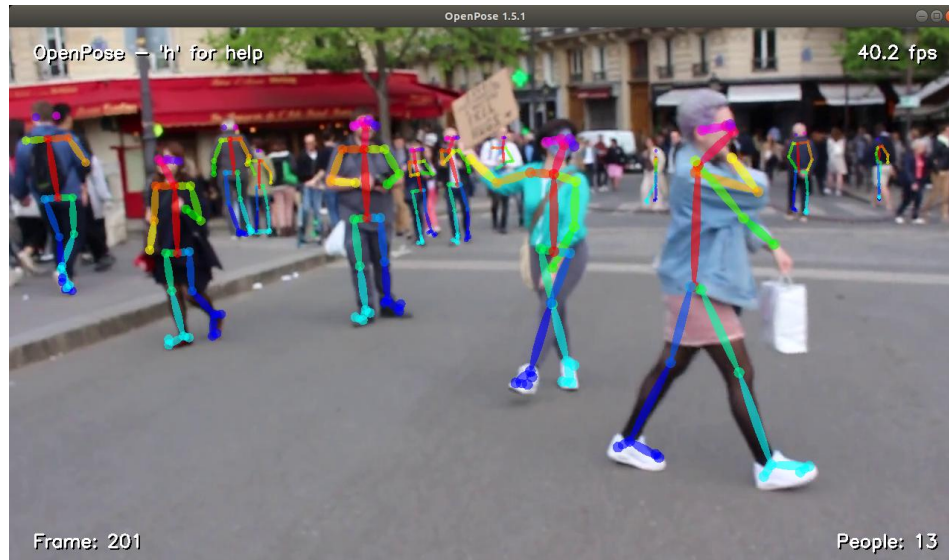
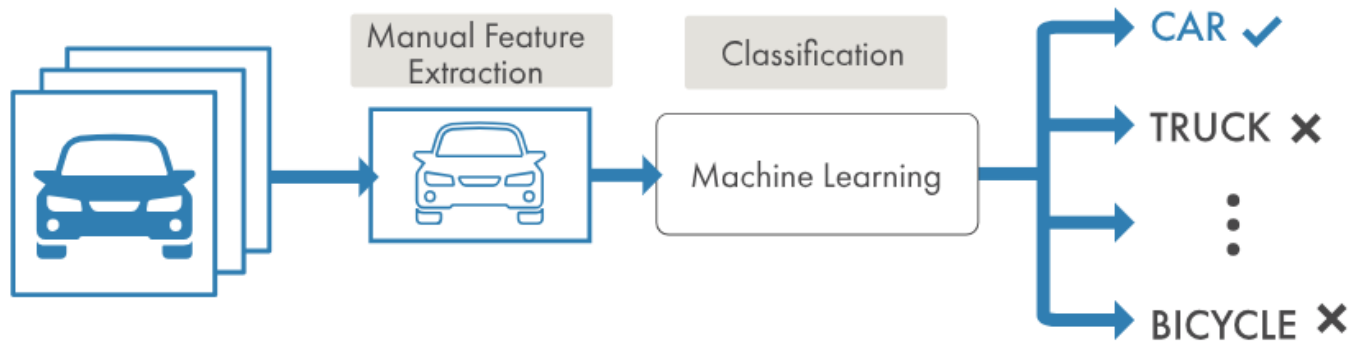


Figure 1: OpenPose pipeline.

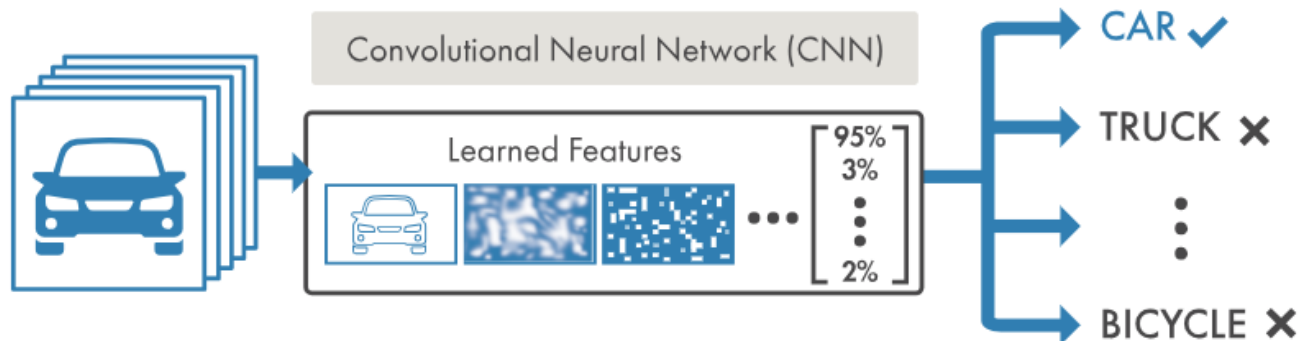
Object Classification/Detection

Classical Machine Learning V.S. Deep Learning

MACHINE LEARNING

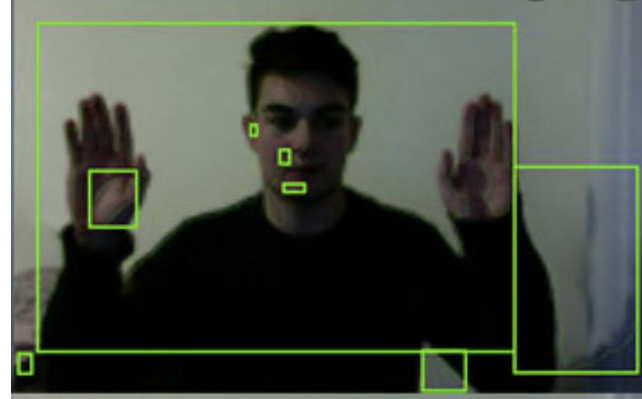


DEEP LEARNING



A third option: Hybrid solutions

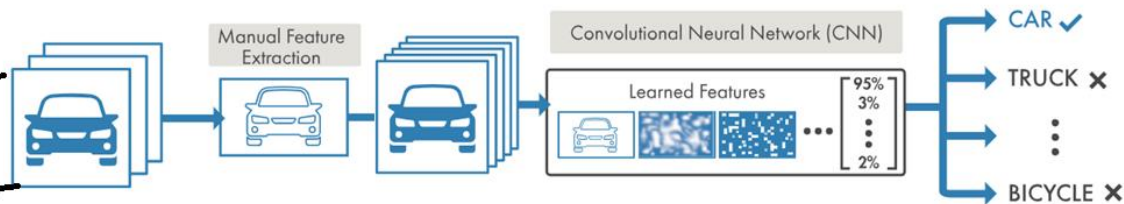
Starting from classical
blob detectors/segmentation
/block processing



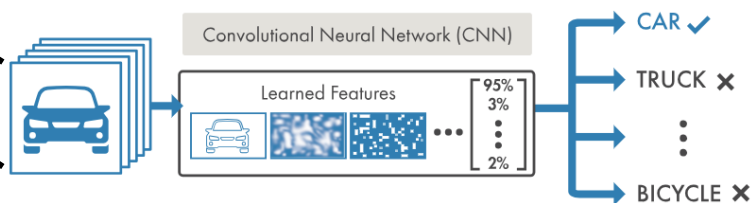
Blob detectors:
where the image responds
to specific kernel convolution
(see “similarity” lessons)

Classical feature extr.

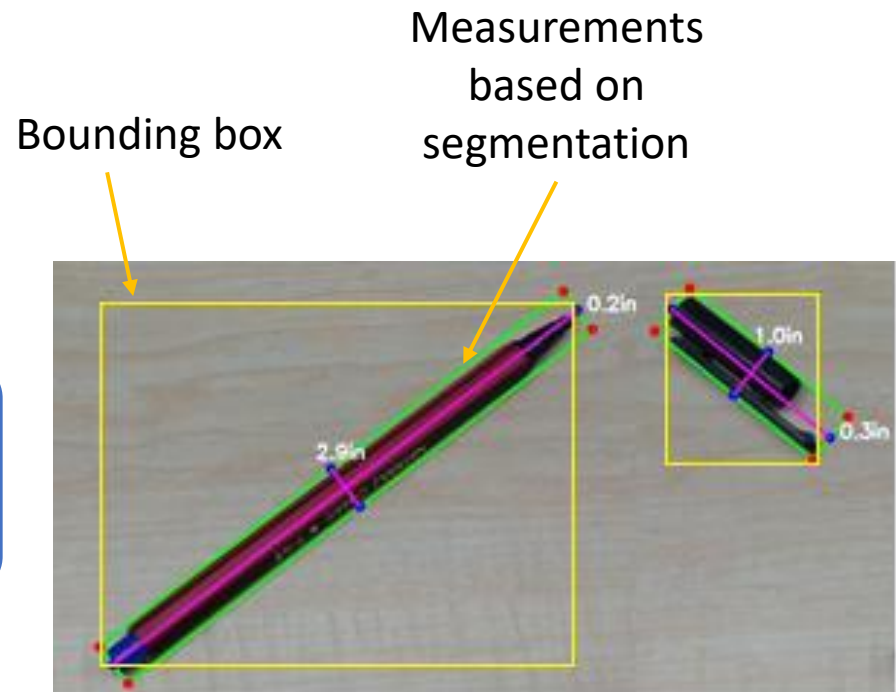
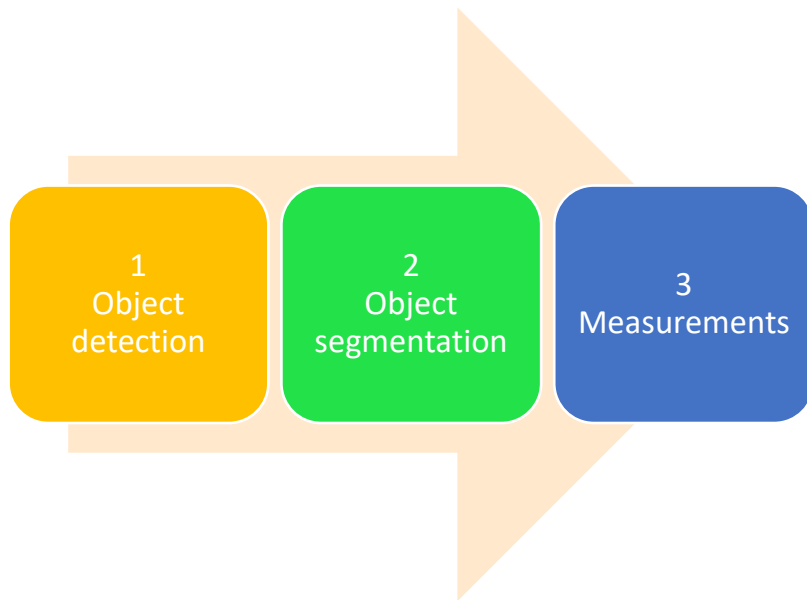
DEEP LEARNING



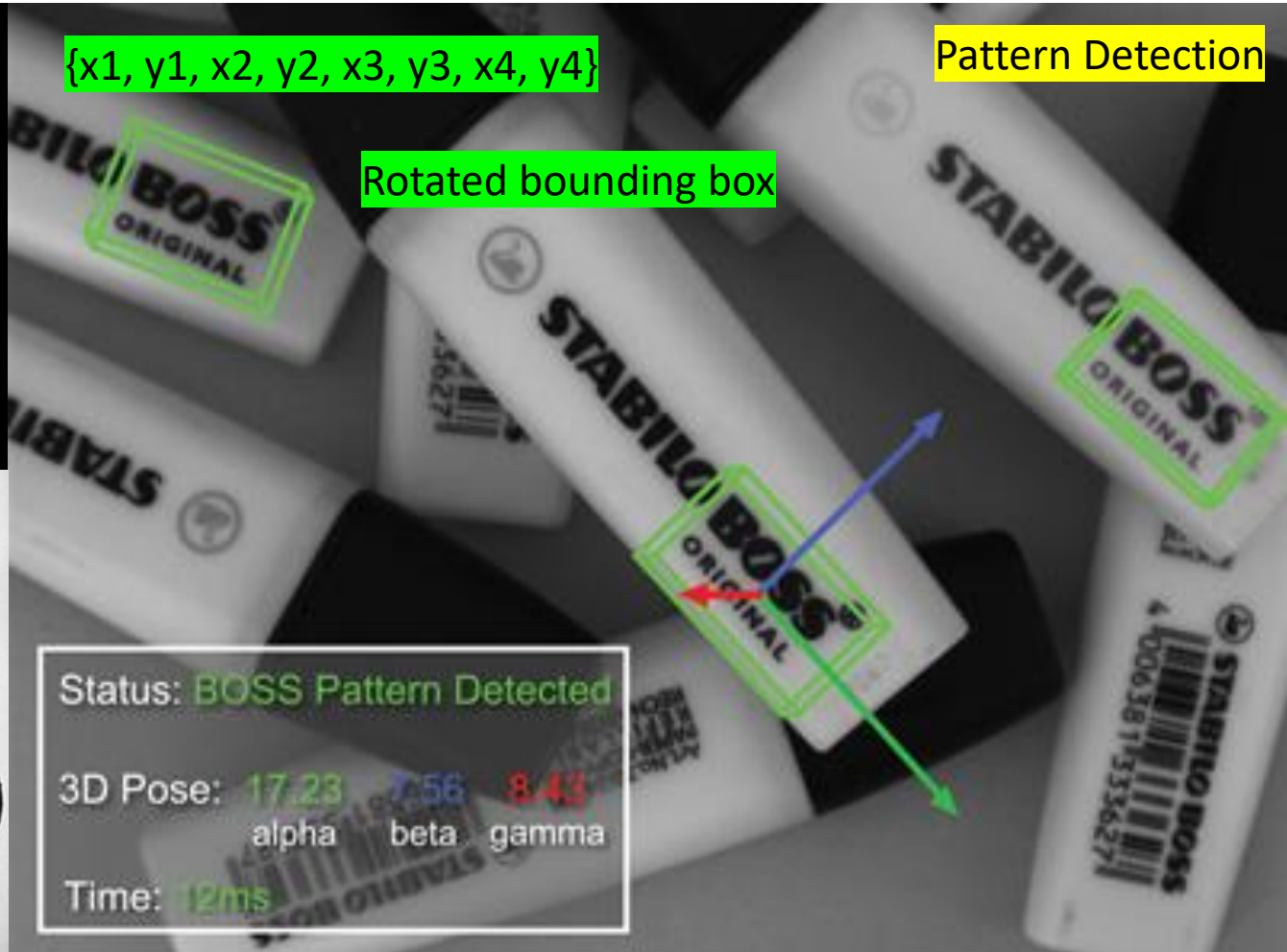
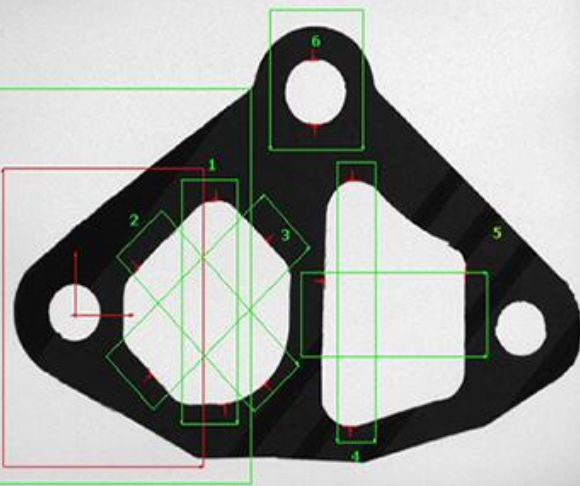
DEEP LEARNING



Object Measurement: a three step process



Object measurements in Industrial applications



Measurements with traditional pattern matching

Object segmentation and measurement: Bin picking

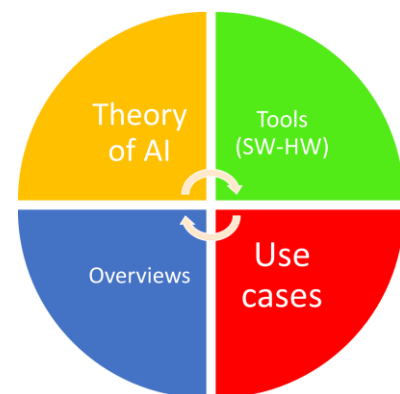


Processing the exact grabbing position with intelligent vision system

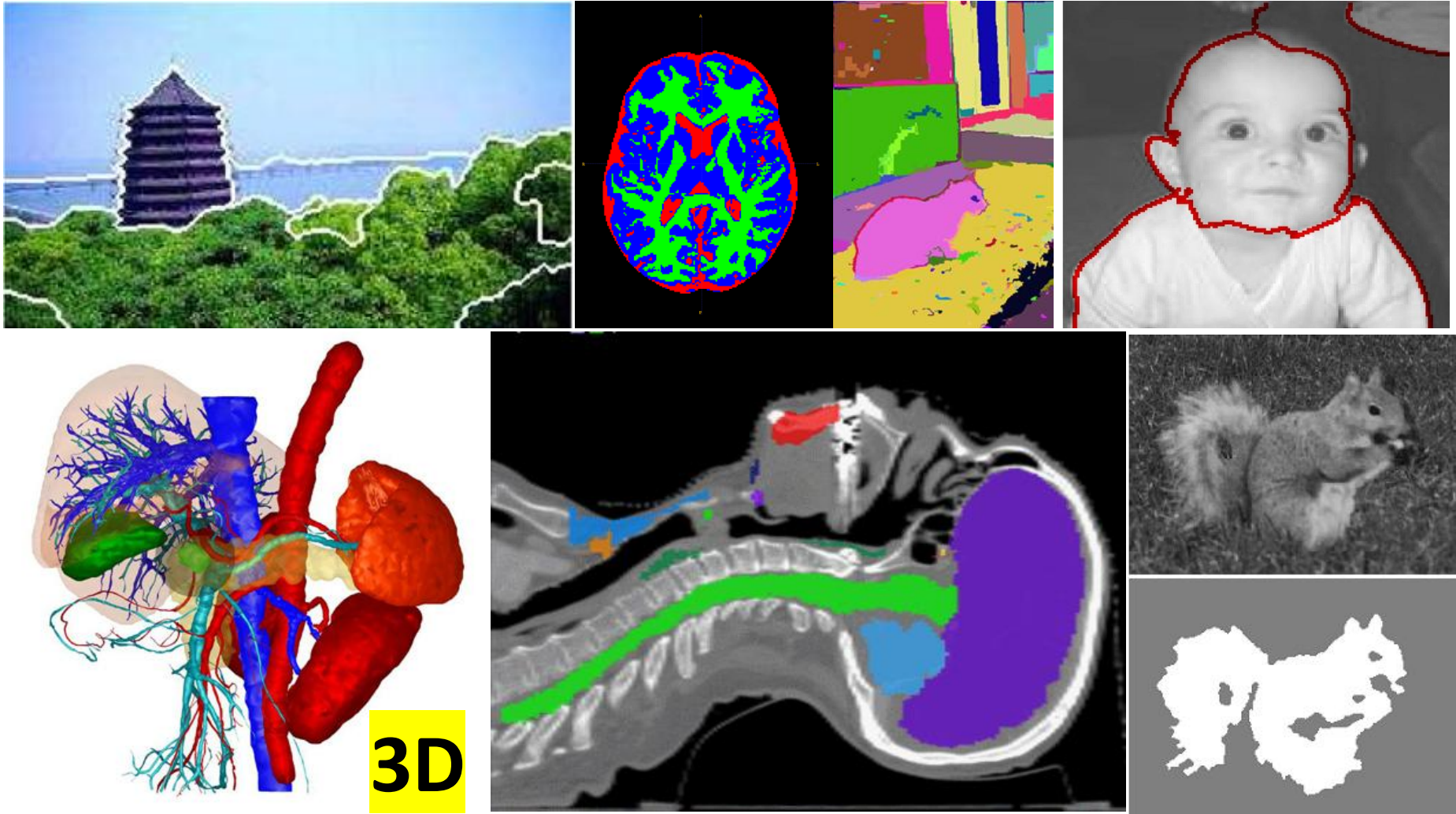


THEORY

Object segmentation



General segmentations



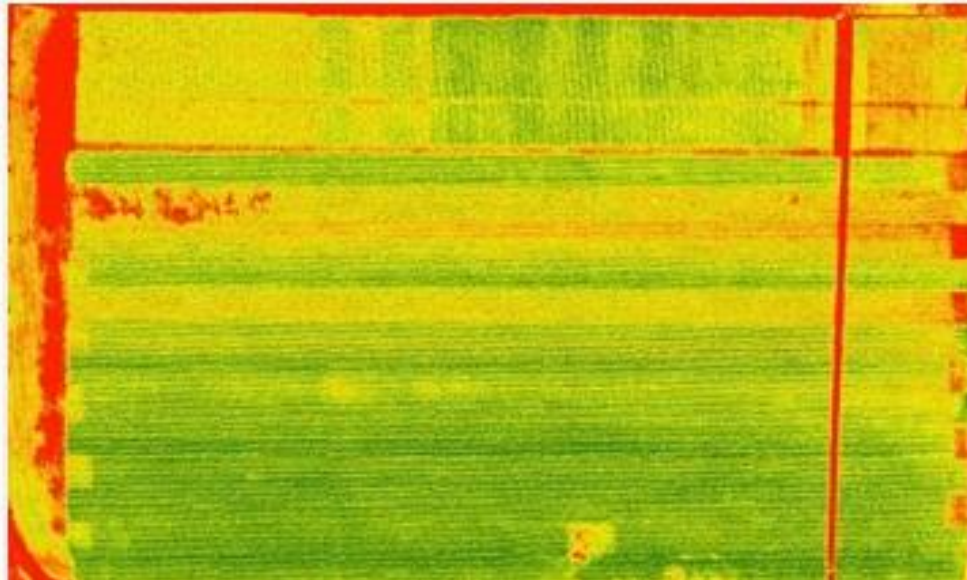
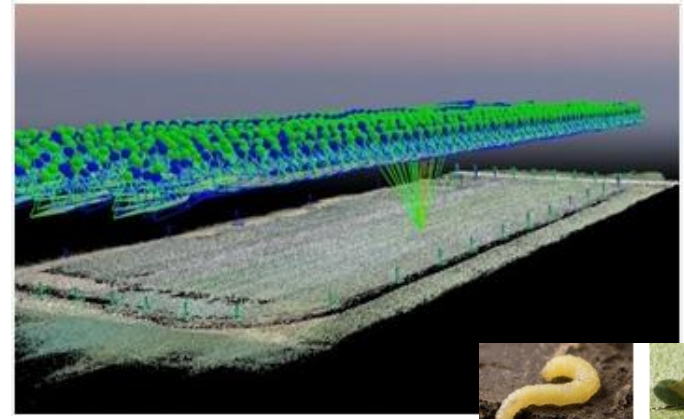
From 3D data it's possible to segment volumes of the objects

Ex. in environmental applications: Drones



frames

Segmentation and measurements



- Example of an outbreak of Corn rootworm.
- The static images and video would help the farmer or crop service vendor identify generally where the damage is occurring.
- Integration with Geographic Information Systems (GIS).

Ex. in environmental applications: Satellites and airborne



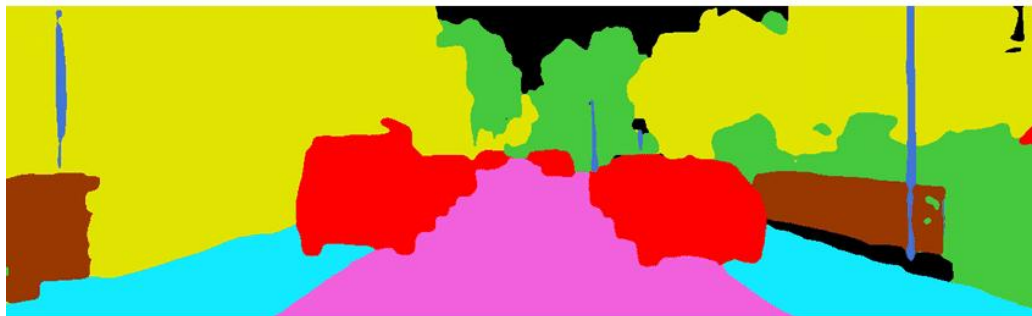
Multiband

Intelligent
Vision
Systems

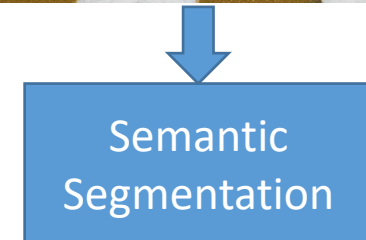
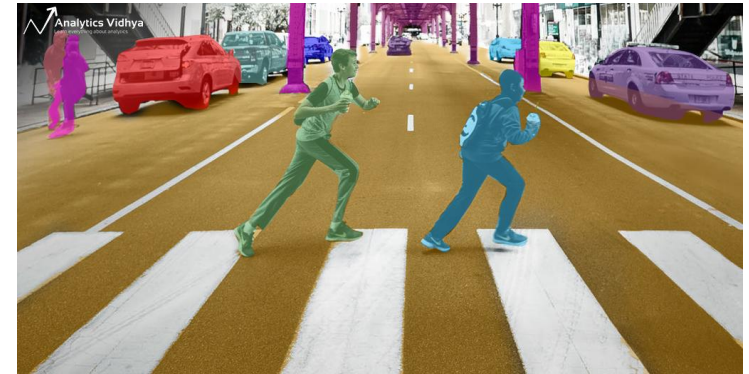
Segmentation and
measurements



Ex. Autonomous vehicles



 Road	 Sidewalk	 Building	 Fence
 Pole	 Vegetation	 Vehicle	 Unlabel



Pedestrians!! →
Big area! →
They are close!! →
Break!!

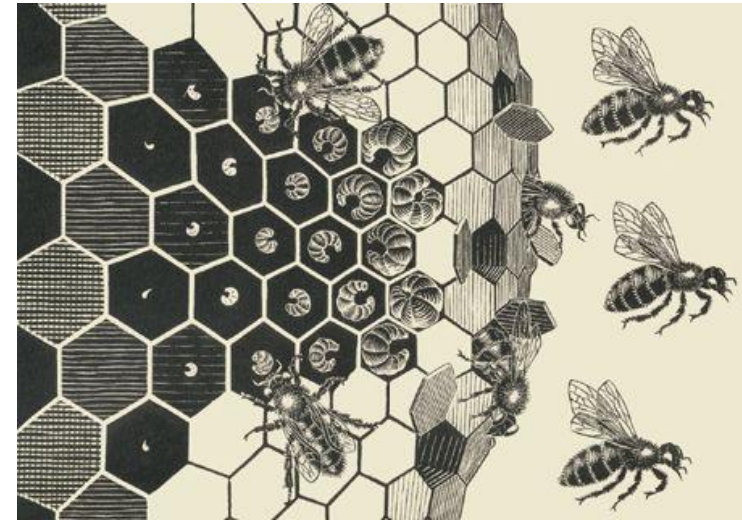
A few more examples of applications of image segmentation ...

- **Medical Imaging**
Image segmentation is used to delineate structures in medical images (e.g., tumors, organs, or blood vessels) for diagnosis or surgical planning.
- **Object Recognition**
By segmenting an image into its constituent parts, algorithms can better recognize and classify objects within the image.
- **Face Recognition**
Segmentation is used to isolate faces from an image's background, improving the accuracy of face recognition systems.
- **Video Surveillance**
Image segmentation helps in tracking objects or people over successive frames in surveillance videos.
- **Document Analysis**
It's used to separate text from backgrounds in scanned documents or in OCR (Optical Character Recognition) applications.
- **Biometrics**
In fingerprint recognition or iris recognition, image segmentation is used to isolate the relevant features.
- **Augmented Reality**
It is used to separate foreground objects from the background in real-time, to overlay digital content onto the real world.
- **Computer Graphics**
Image segmentation is used to edit images or videos, such as changing the background of an image.
- **Wildlife Tracking**
Image segmentation is used to identify and track wildlife in images or videos for research and conservation efforts.
- **Industrial Inspection**
Image segmentation is used in quality control systems to identify defects in manufactured products.
- **Fashion Industry**
Segmentation is used in virtual try-on applications where clothing items need to be segmented and superimposed on the user's image.
- **Robotics**
to recognize and interact with objects in their environment.

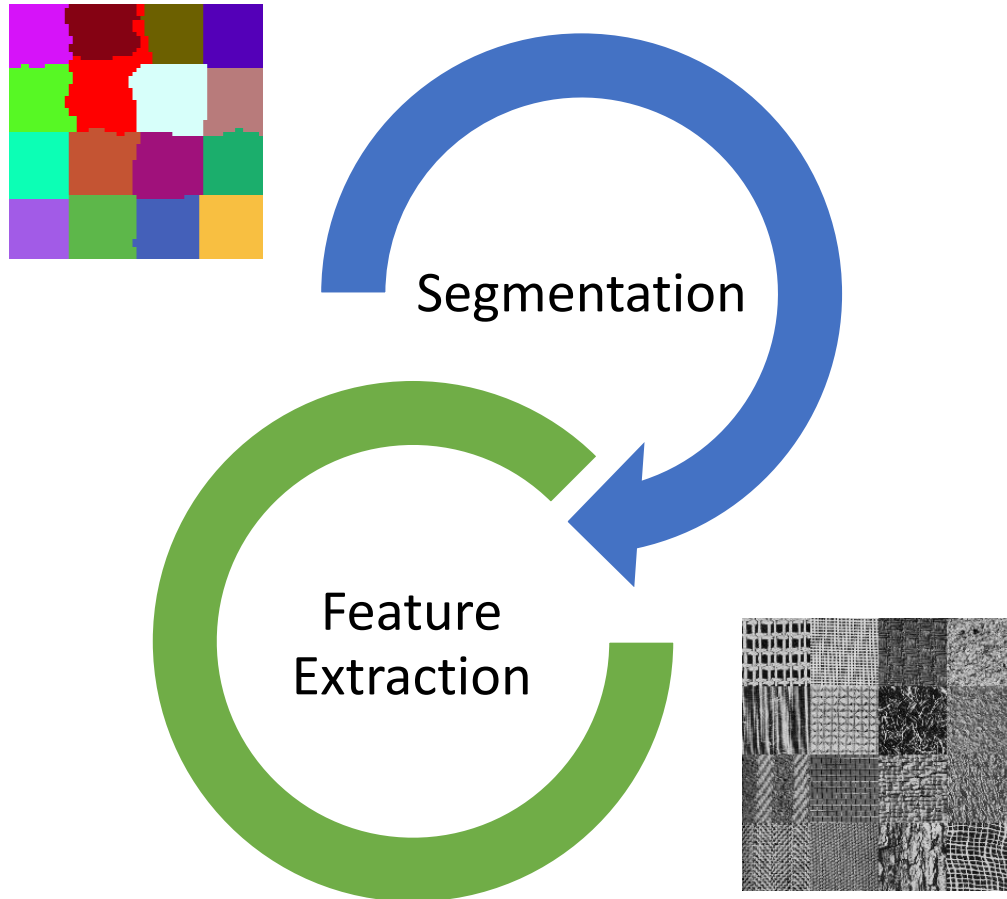


Segmentation “definition”

- Segmentation
 - Split or separate an image into regions
 - To facilitate recognition, understanding, and Region Of Interests (ROI) processing
- Ill-defined problem
 - The definition of a region is context-dependent



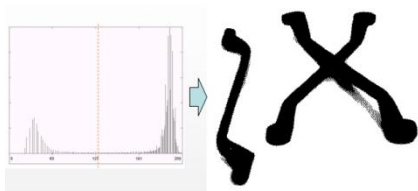
Two close tasks



Basic taxonomy

Traditional
Segmentation
methods

Global knowledge,
(using histogram
image features)

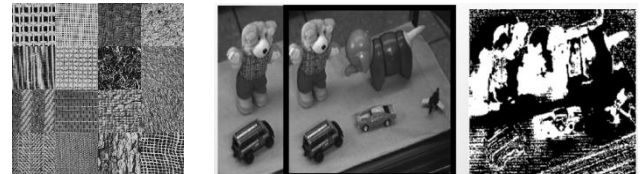


Edge-based
segmentations



Region-based
segmentations

Brightness, texture,
velocity field,
color, ..

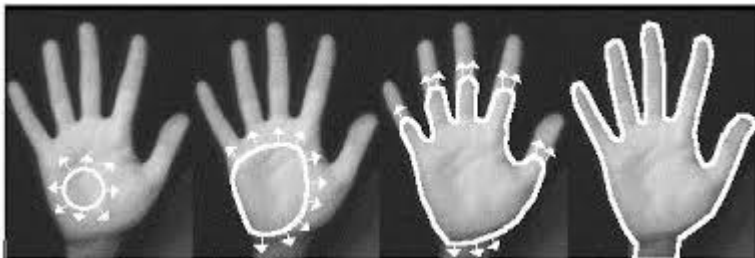


Classical methods and other met.

(for your reference in case of need)

CLASSICAL METHODS

- Global knowledge
 - Thresholding
 - Histogram-based methods
 - Compression-based methods
- Edge detection
- Region-growing methods
 - Active Contour Models - “Snakes”
 - Watershed transformation
 - Model based segmentation
 - Deformable models (Snakes)



An example of a region-growing method (Snake)

OTHER METHODS

- Split-and-merge methods
- Clustering methods (color, intensity, texture, and sometimes spatial coordinates)
 - K-means clustering
 - Gaussian Mixture Models (GMM)
 - Mean Shift clustering
- Partial differential equation-based methods
- Graph partitioning methods
- Multi-scale segmentation
 - One-dimensional hierarchical signal segmentation
 - primal sketch
- Semi-automatic segmentation
- Neural networks segmentation
 - Deep Learning (next slides)

Image segmentation via k-means clustering (unsupervised)

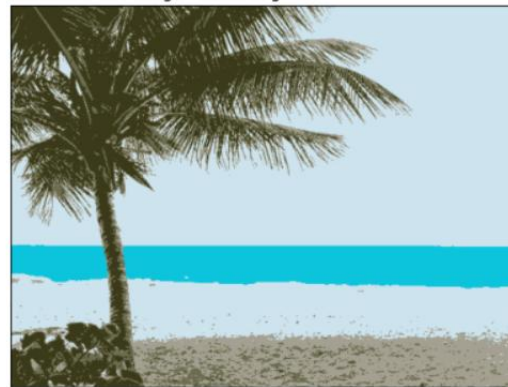
- K-means clustering can be used to segment images by grouping pixels based on their color similarity, resulting in distinct color-based regions within the image.

Very simple, but
oversegmentation or
undersegmentation
can occur

Original Image



Segmented Image when K = 5



Input Image: cameraman



segmented Image: cameraman

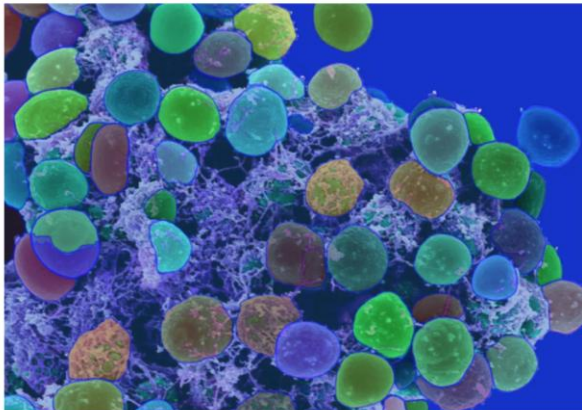


Recent improvements:

Segment Anything Model (SAM)

A deep learning method, by Meta AI

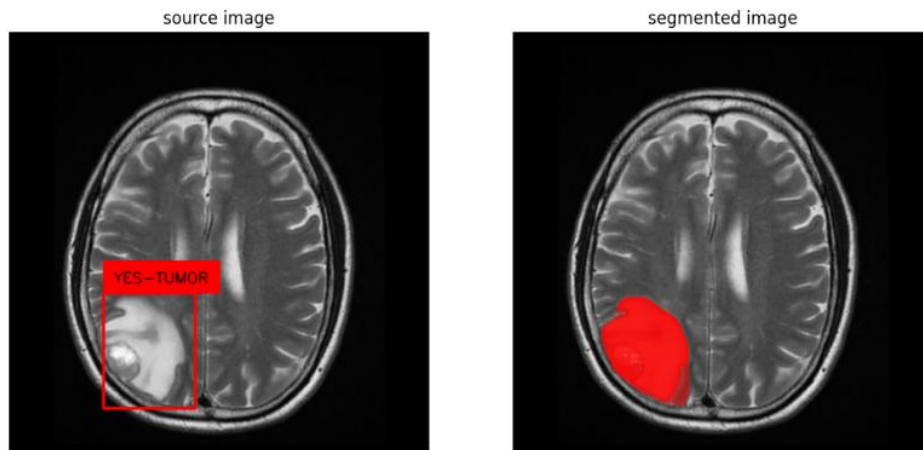
- Main features
 - A **promptable** segmentation system
 - with **zero-shot generalization** to unfamiliar objects and images, without the need for additional training (11M images, 1B+ masks)
- SAM has learned a general notion of what objects are → this understanding enables zero-shot generalization
- Can generate a segment a frame in about 50 ms on a PC



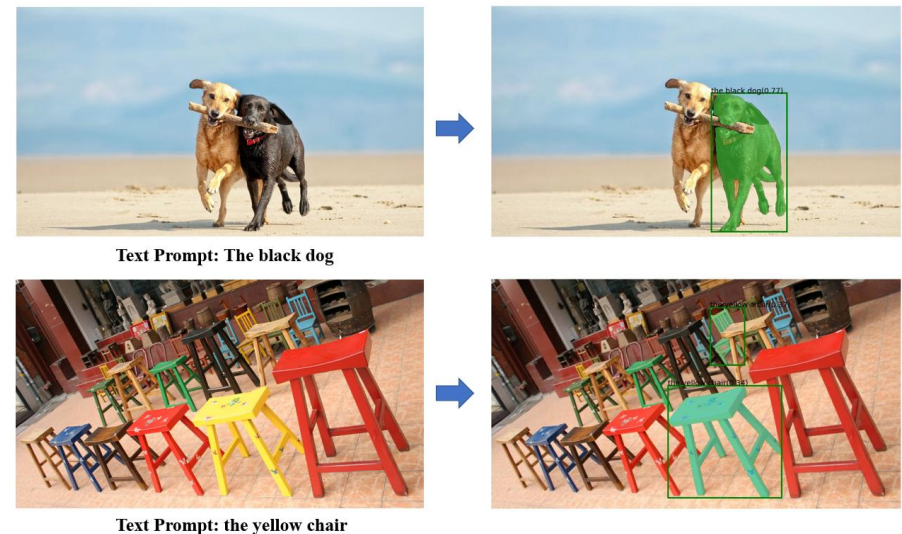
SAM

Promptable segmentation

- Researchers from Meta AI trained SAM to return a valid segmentation mask for any prompt
 - foreground/background **points**, a rough box or **mask**, freeform **text**, or, in general, any information indicating what to segment in an image.



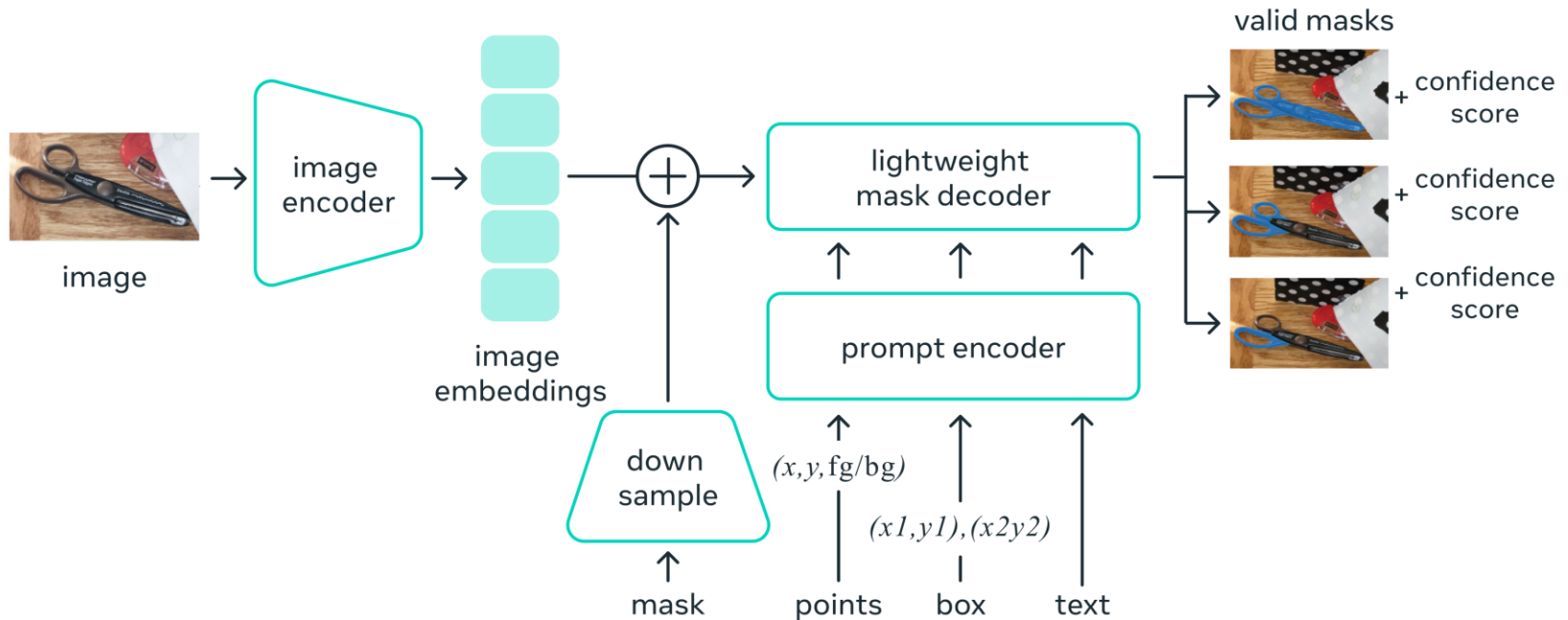
Input (prompt) is the bounding box



SAM

Promptable segmentation

Universal segmentation model



Server side

Client side (e.g., cellphone)

SAM in colab with metaseg

✓
5 s

```
[2] # installed the metaseg library in the Colab environment.  
!pip install metaseg
```

✓
0 s

```
# upload an image to your Google Colab environment using the file browser on the left.  
# click on the folder in case to open the menu  
# In this example, we'll be using an image named puppies.jpg  
from IPython.display import Image  
Image("puppies.jpg")
```



```
[10] # import the segmenter
from metaseg import SegAutoMaskPredictor, SegManualMaskPredictor
```

```
results = SegAutoMaskPredictor().image_predict(
    source="puppies.jpg",
    model_type="vit_1", # vit_1, vit_h, vit_b
    points_per_side=8,
    points_per_batch=64,
    min_area=0,
    output_path="output.png",
    show=False,
    save=True,
)
```

```
# reate an instance of the SegAutoMaskPredictor class and use it to segment the image with the following code:
```

```
vit_1 model already exists as 'vit_1.pth'. Skipping download.
```

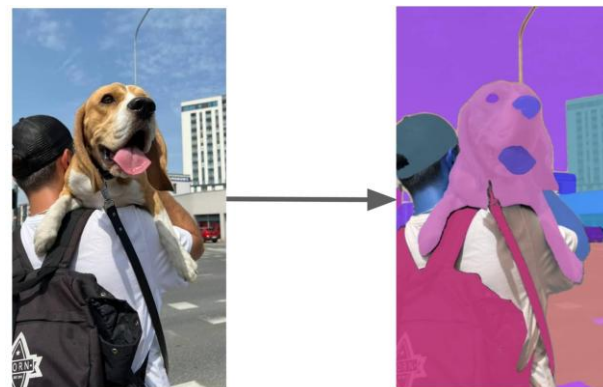
```
[5] # print the output image
from IPython.display import Image
Image("output.png")
```



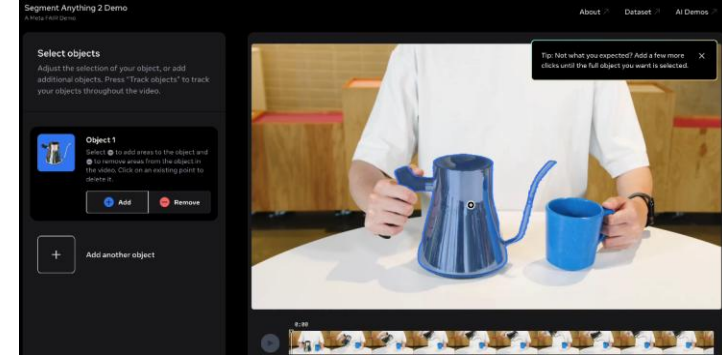
SAM in colab with metaseg

SAM2 (video and images)

SAM 2 represents a significant improvement over the original model. It is the first unified model for real-time image and video segmentation



SAM2 (video and images)

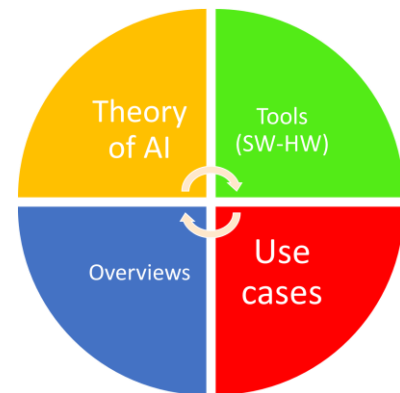


- **Speed:** Processes approximately 44 frames per second, making it suitable for real-time applications.
- **Zero-shot generalization:** Segments unseen objects during training without the need for customization.
- **Video memory:** Includes a memory mechanism to track objects across frames, handling occlusions and complex movements.
- **SAM 2 is 6x faster** and more accurate for image segmentation compared to the original SAM, and it adds video segmentation capabilities that SAM lacks (100-300M param.)
- It is **open-source** (Apache 2.0 license)



THEORY

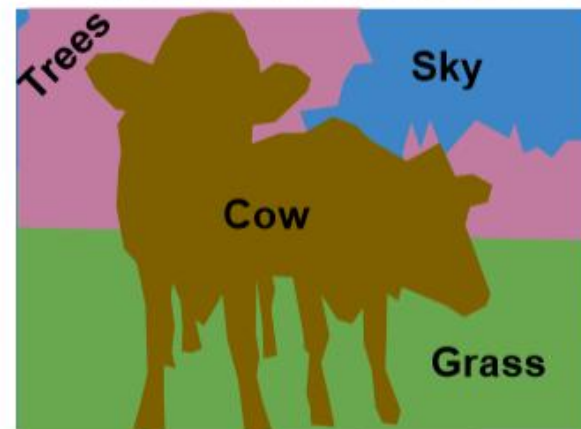
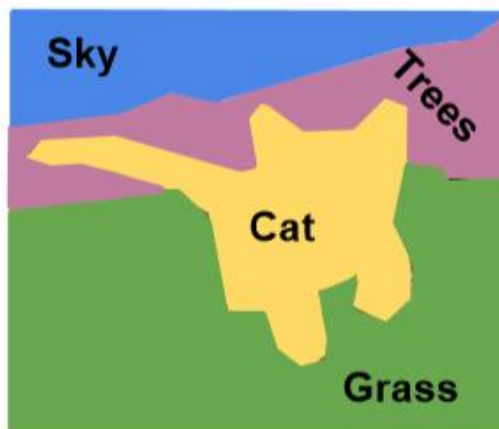
Semantic segmentation



Semantic segmentation (recall)

Label each pixel in the image with a category label

Don't differentiate instances, only care about pixels

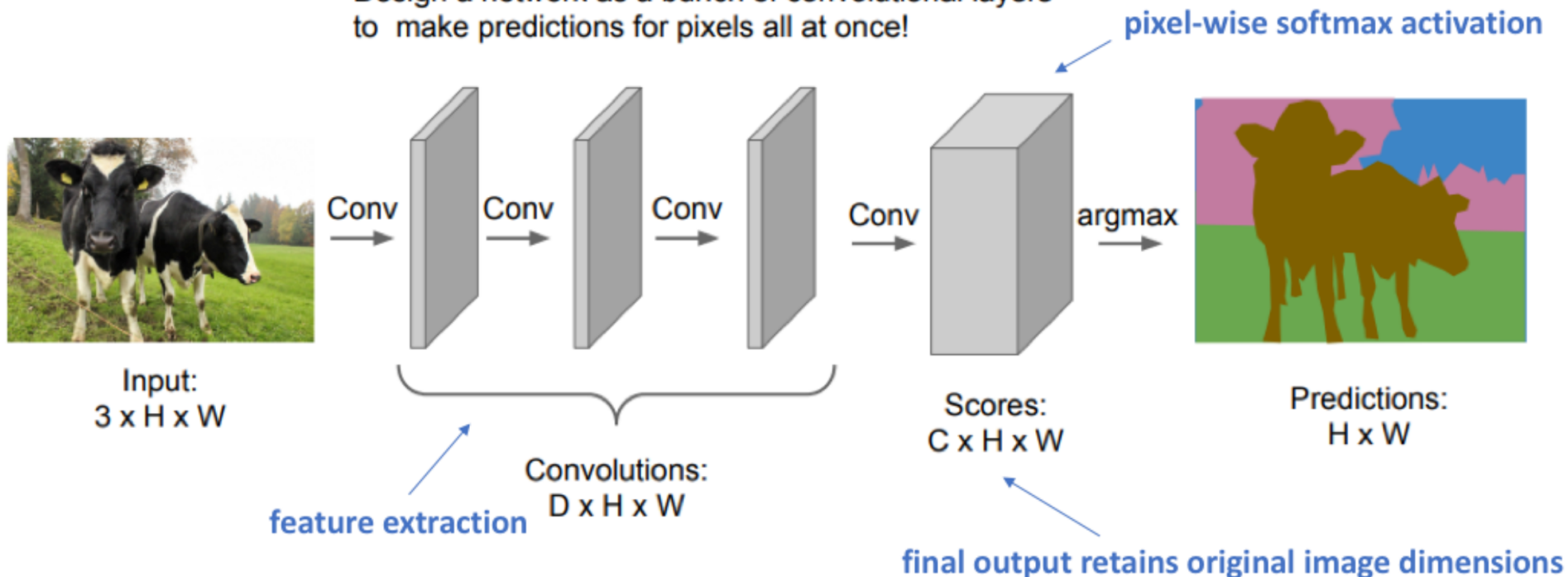


Pixel classification is a good idea but... not feasible

Fully Convolutional Network

Imagine that every single pixel of the output image is classified with a softmax...

Design a network as a bunch of convolutional layers to make predictions for pixels all at once!



Semantic segmentation

A fully convolutional solution

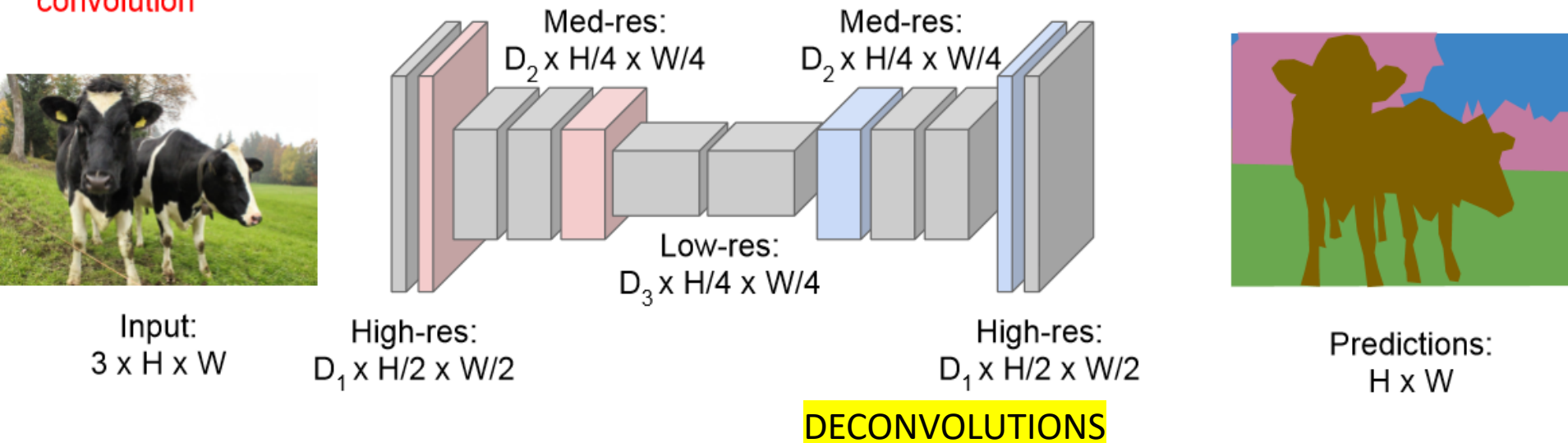
Fully Convolutional Network with Downsampling and Upsampling

Encoder/decoder structure

Downsampling:
Pooling, strided
convolution

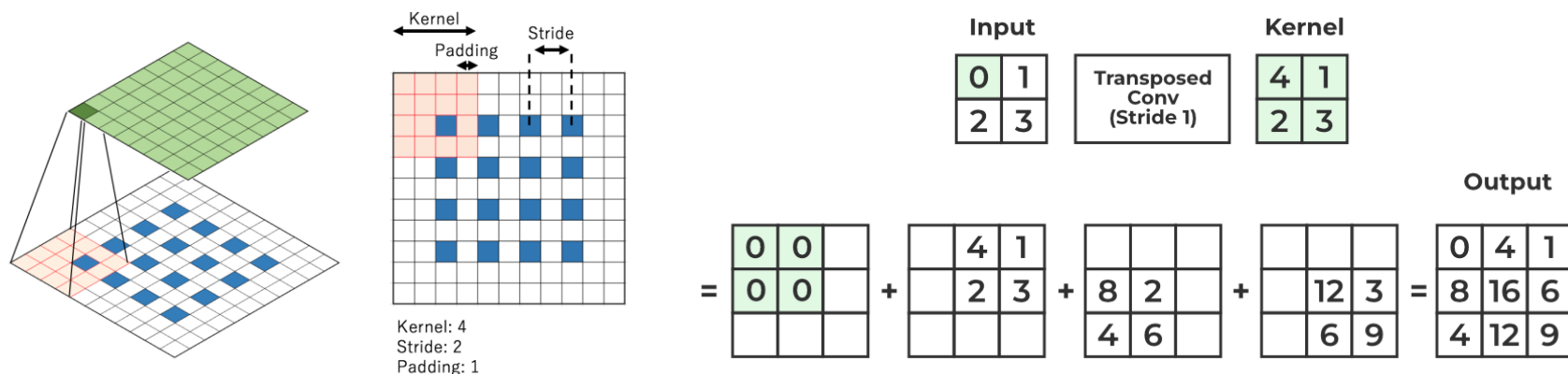
Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Upsampling:
Unpooling or strided
transpose convolution

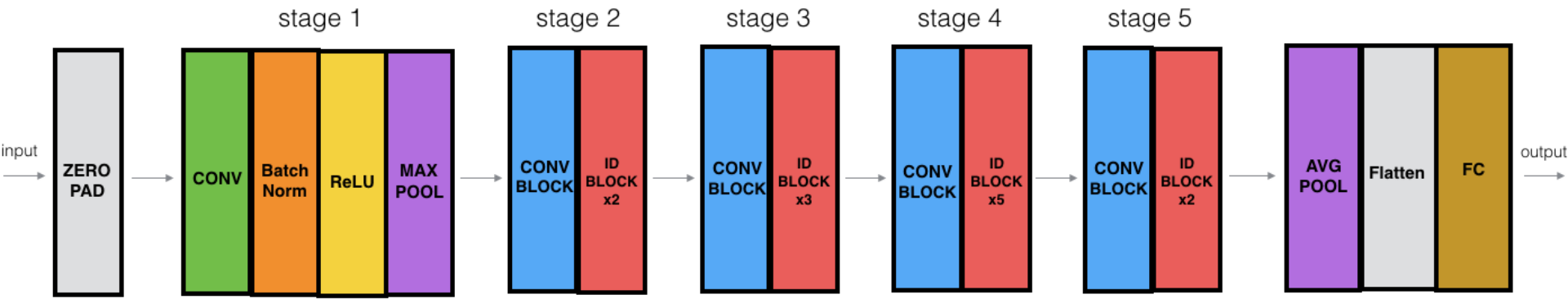


Upscaling `nn.ConvTranspose2d()`

- Used in the decoder sections of deep networks, such as autoencoders or U-Net, to perform upscaling.
- It increases the spatial resolution of feature maps by learning how to reconstruct higher-resolution outputs from lower-resolution inputs.



ResNet-50 for classification

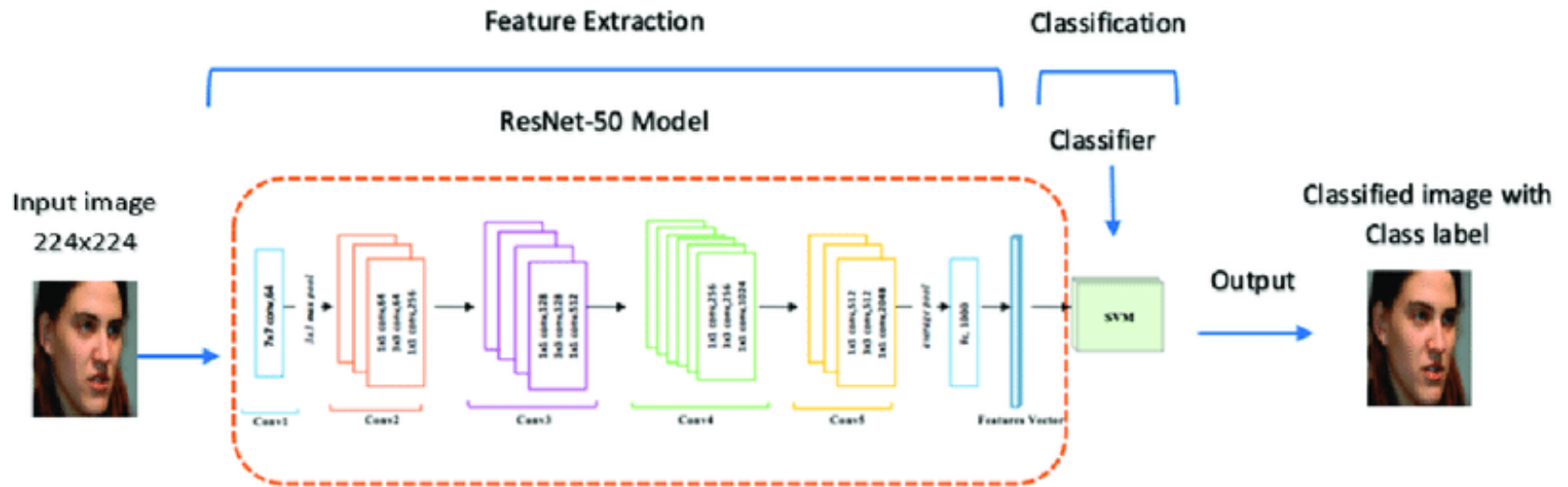


The native network is designed for classification (the output is an integer)!

KERAS: `model.param_count()`
 → **25M parameters**

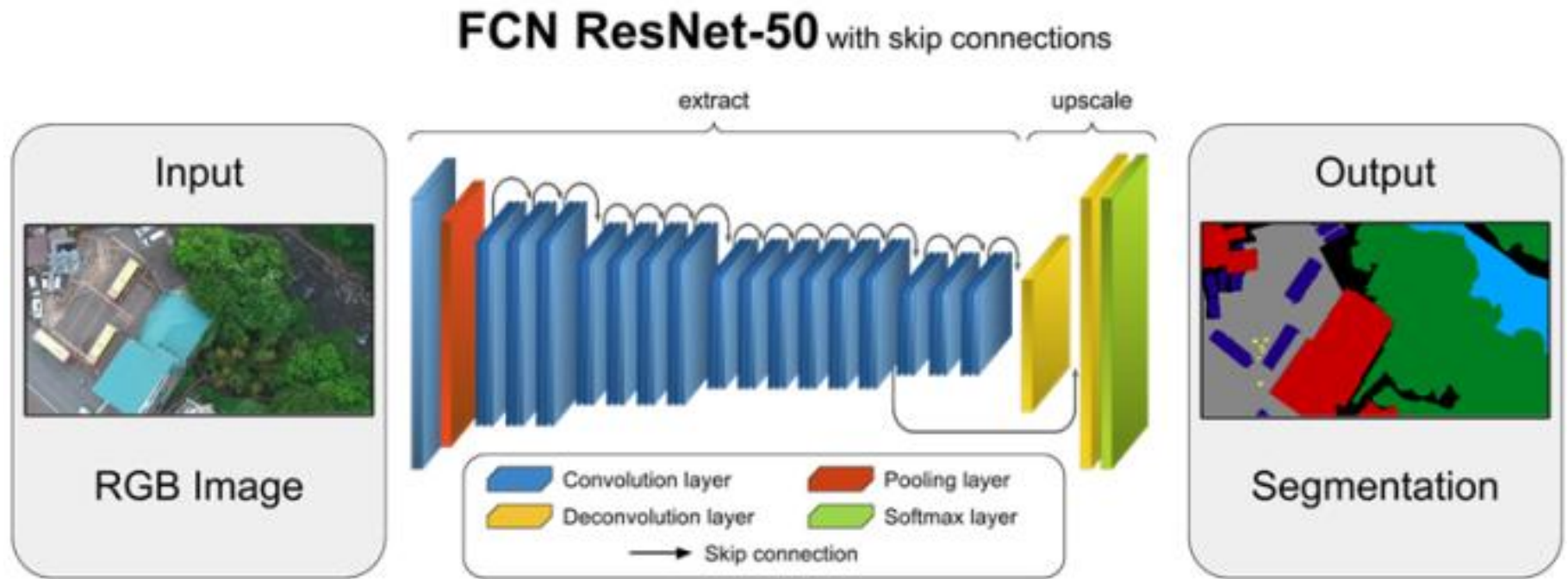
layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

ResNet-50 for feature extraction

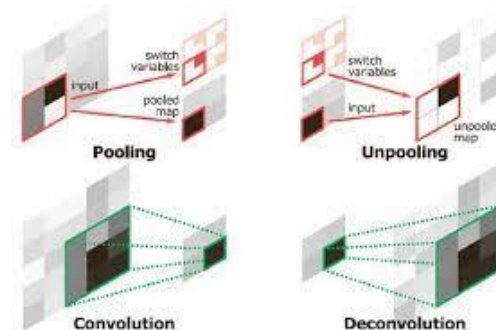


You can use ResNet-50 to process features and then use your classifier to create a new task

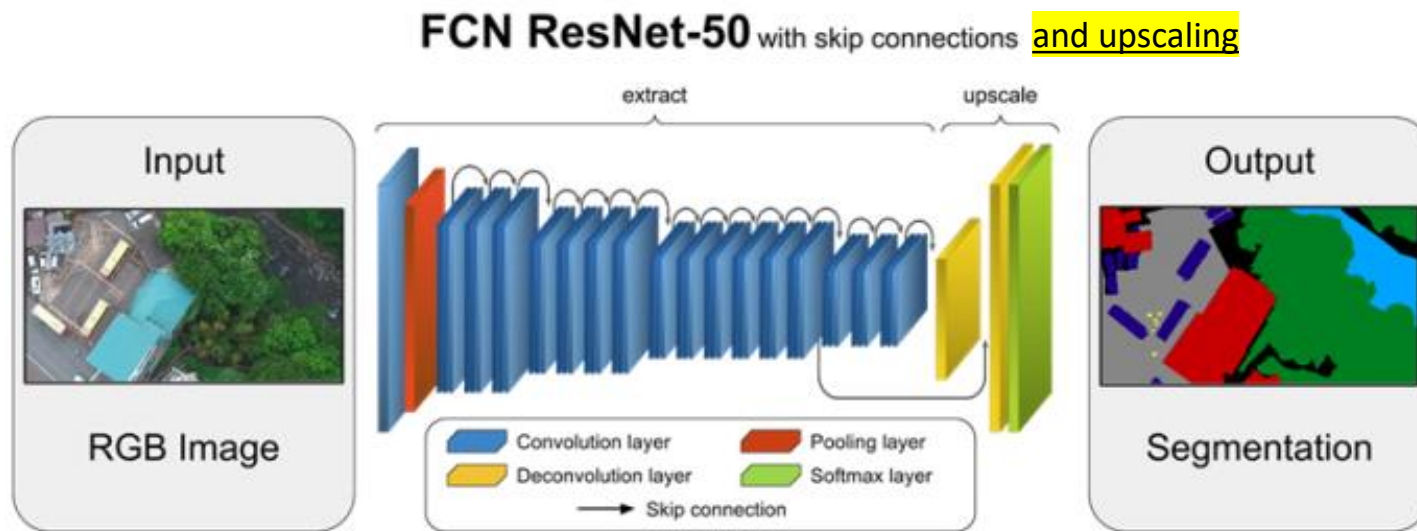
ResNet-50 → ResNet50 + Upscaling trainable semantic segmentation



Transposed convolution is a learnable upsampling operation. It is not the exact mathematical inverse of convolution in general.



ResNet-50 with upscaling for classification? NO!



After adding an upsampling decoder, the network becomes a segmentation architecture, producing an image-like output, not a class ID.

Image classification via DL

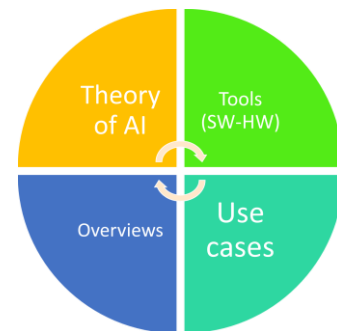
Model	Year	# Parameters	Top-1 Accuracy (ImageNet)	Notes
VGG-16 / VGG-19	2014	138M / 144M	~71.5%	Simple but heavy; no skip connections
ResNet-50	2015	25.6M	~76.0%	Introduces residual blocks for deeper training
Inception-v3	2015	23.8M	~78.8%	Efficient multi-path convolutional blocks
EfficientNet B0–B7	2019	5.3M – 66M	77.1% – 84.4%	Scalable CNN using compound scaling
EfficientNetV2 S–L	2021	24M – 120M	84.0% – 87.3%	Faster training, better accuracy and efficiency
Vision Transformer (ViT-B)	2020	86M	~85.0%	Transformer-based model using patch embeddings
ConvNeXt (Base)	2022	89M	~85.8%	CNN redesigned with transformer-like components
ConvNeXt V2	2023	3.7M – 650M	76.7% – 88.9%	Uses masked autoencoding; state-of-the-art accuracy
MobileNetV3 / V4	2019 / 2024	~5.4M / TBD	~75% / TBD	Optimized for mobile and edge devices



Use cases

Objects recognition from UAVs/drones

Unmanned Aerial Vehicle (UAV)



Drones/UVAs in applications



Amazon Prime Air

Amazon's drone delivery service is operational in select U.S. cities like Lockeford, California, and College Station, Texas, where customers can receive packages within an hour. Started in UK in 2025..

Used since 2022 in USA

Weights less than 5 pounds (2.26 Kg) and will take less than an hour.

Amazon has received Part 135 air carrier certification from the Federal Aviation Administration.



<https://www.youtube.com/watch?v=ahkeyw-FUIQ>

Objects recognition from UAVs

Let's design a real-time traffic control drone



Landing mark



Traffic control

TRAINING

Training Image



Training Images Selection



Training Images Extraction

car



Feature Extraction CNN

car



nocar



nocar



Classifier Training

SVM

Classifier Testing

Testing Image



DEPLOYMENT

Image Segmentation Mean-Shift



Remove large regions



Feature Extraction CNN



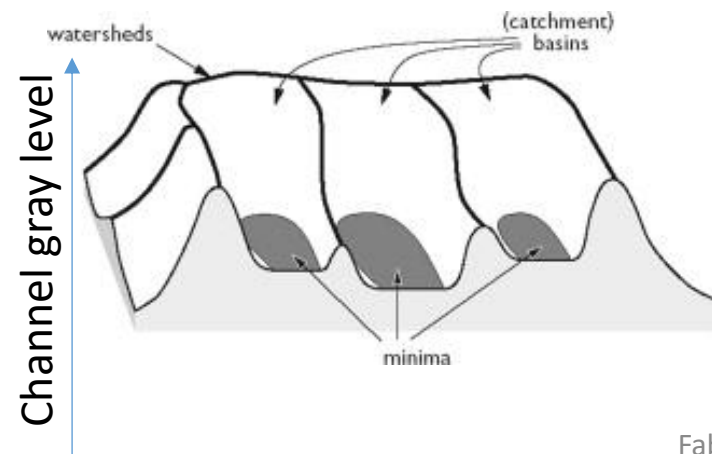
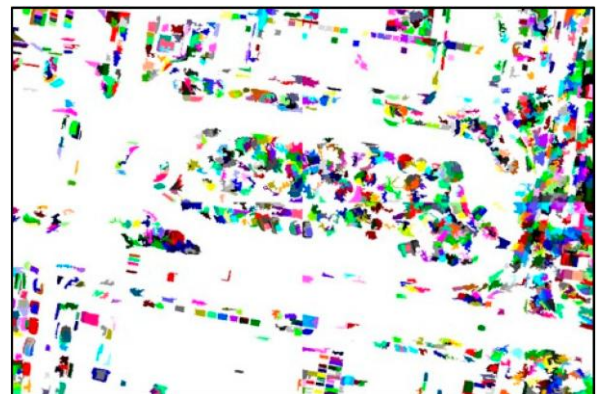
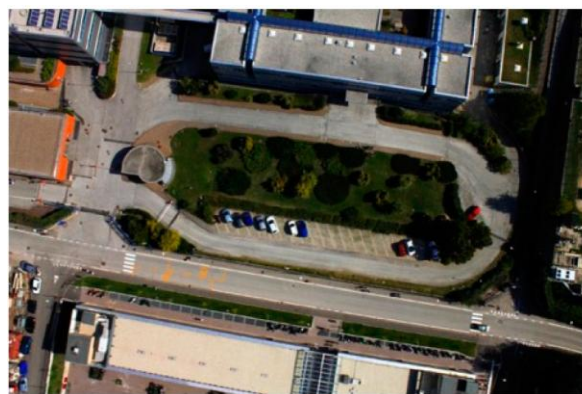
Fine-tuning



TRADITIONAL SEGMENTATION

Obj. Rec. from UAVs: STEP 1 - segmentation

TRADITIONAL SEGMENTATION FOR CANDIDATES

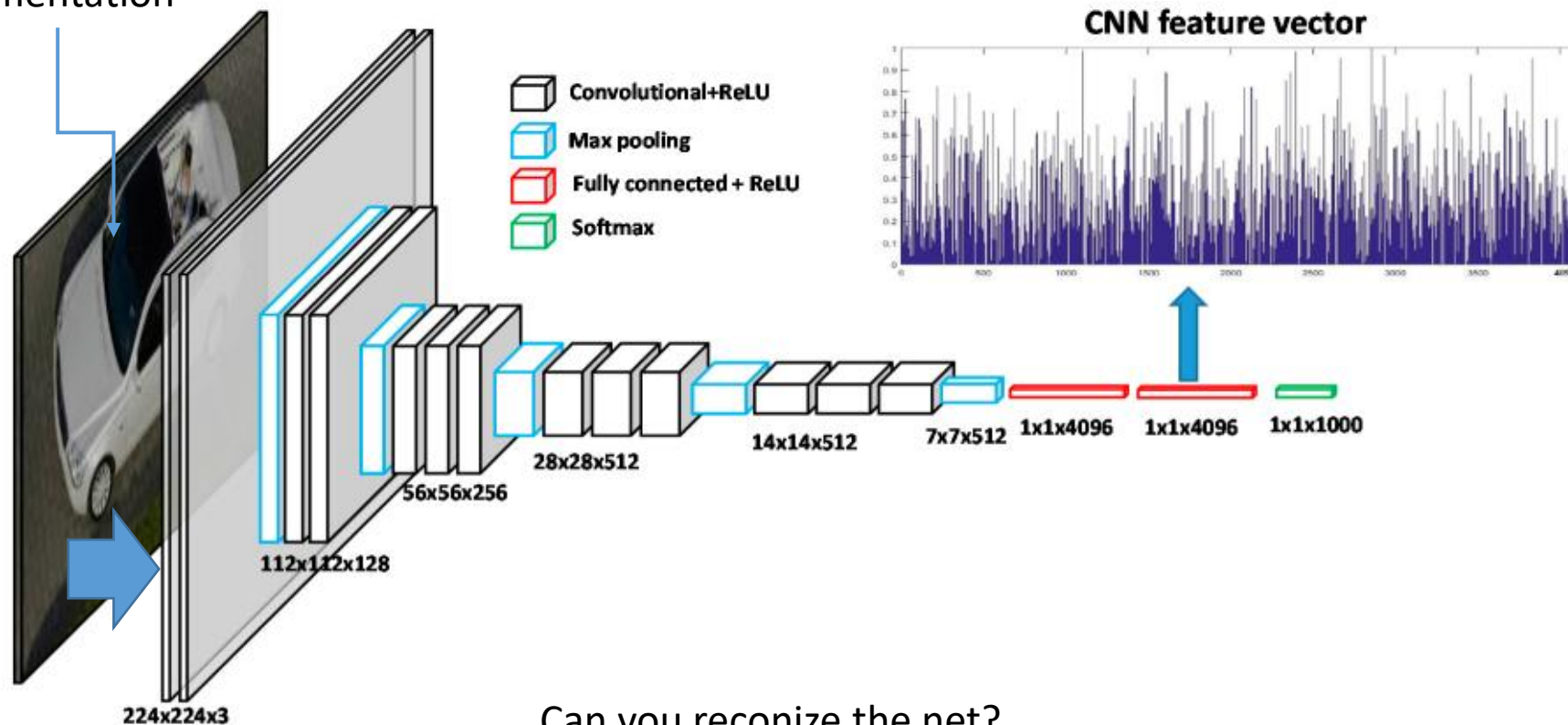


Example:
Watershed Segmentation

Obj. Rec. from UAVs: STEP 2 - CNN design

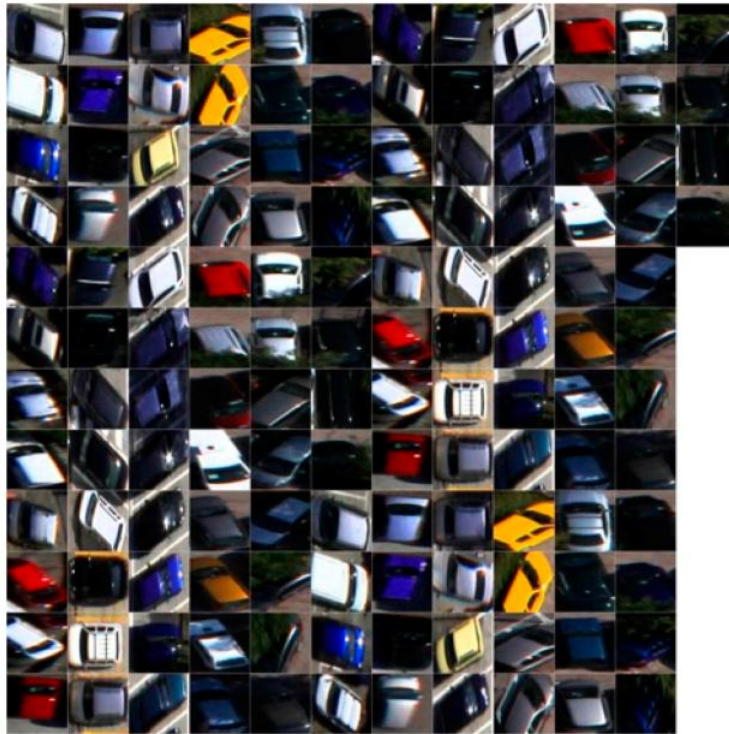
You can use tradition
classifier like SVM or
deep models

From
segmentation

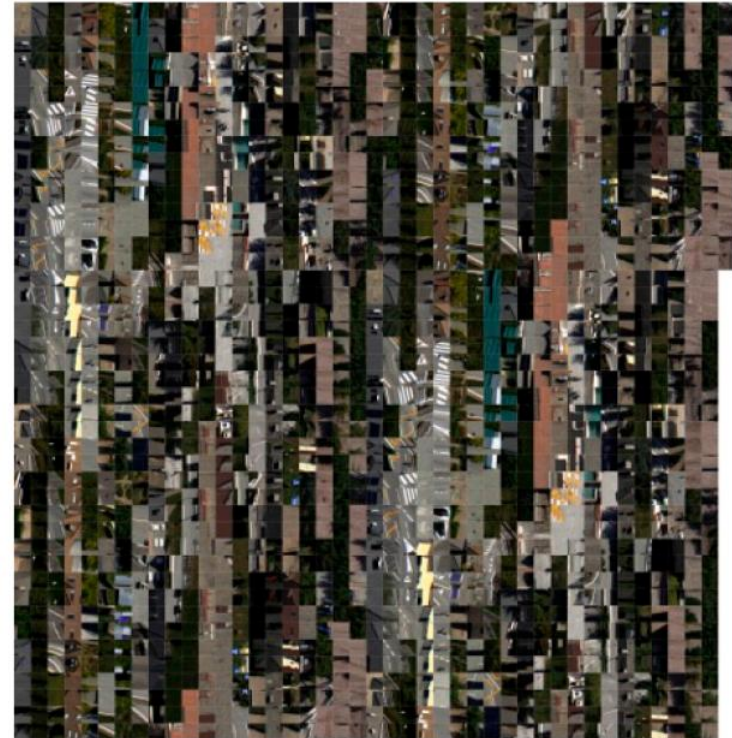


Obj. Rec. from UAVs: CNN learning dataset creation

FINE TUNING

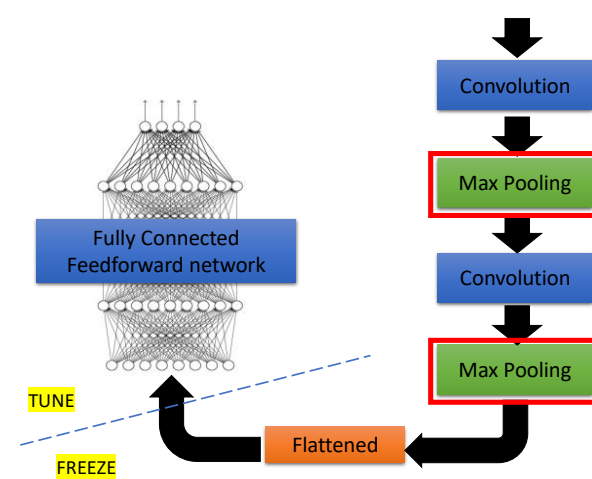


CARS

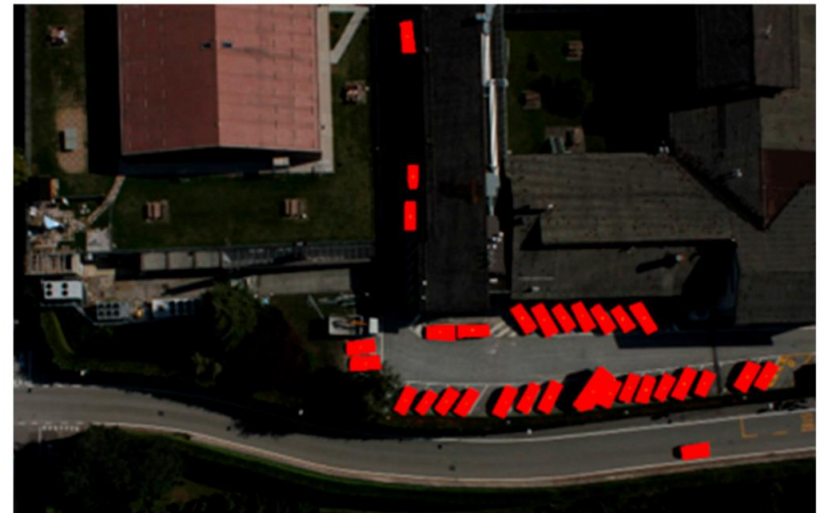
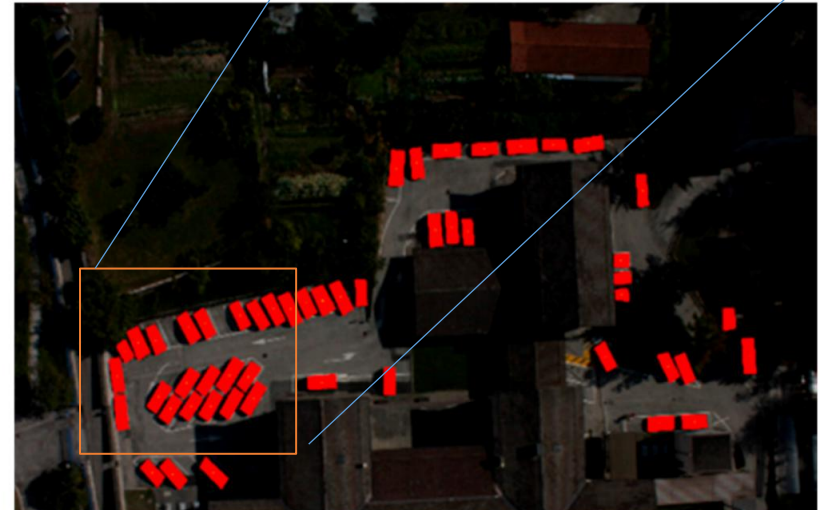
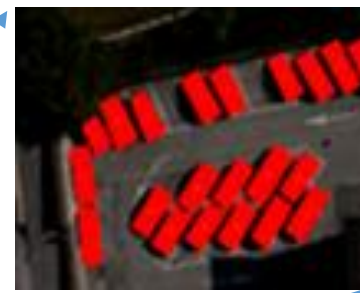


NO CARS

(just a portion)



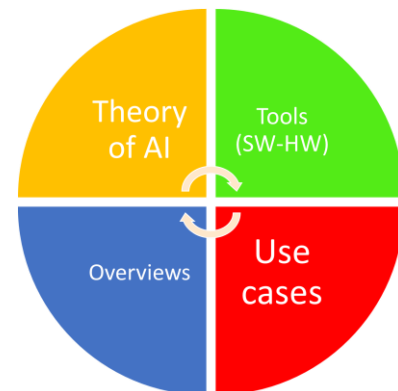
Obj. Rec. from UAVs: results





THEORY

Machine Learning HW

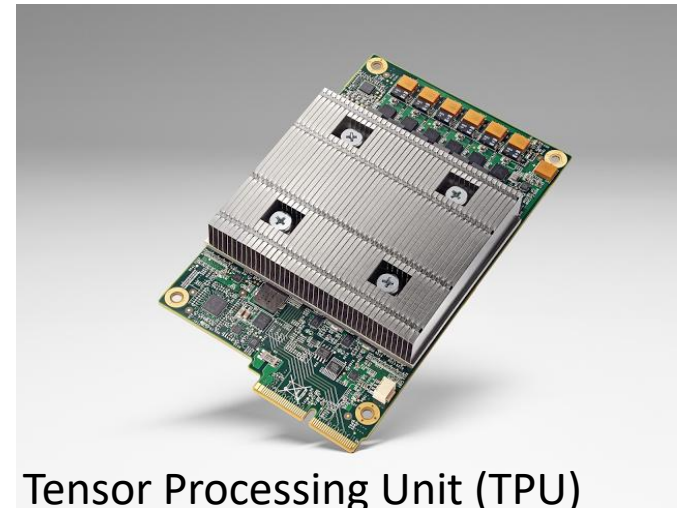


Why Machine Learning Is Possible?

- Mass Storage
 - More data available (TB HD, data centers,..)
- Higher Performance of Computer
 - Larger memory in handling the data
 - Greater computational power for calculating and even online learning
- Examples: NVIDIA RTX / A100 / H100 / Jetson / B200;
Apple Neural Engine / Intel Core Ultra NPU



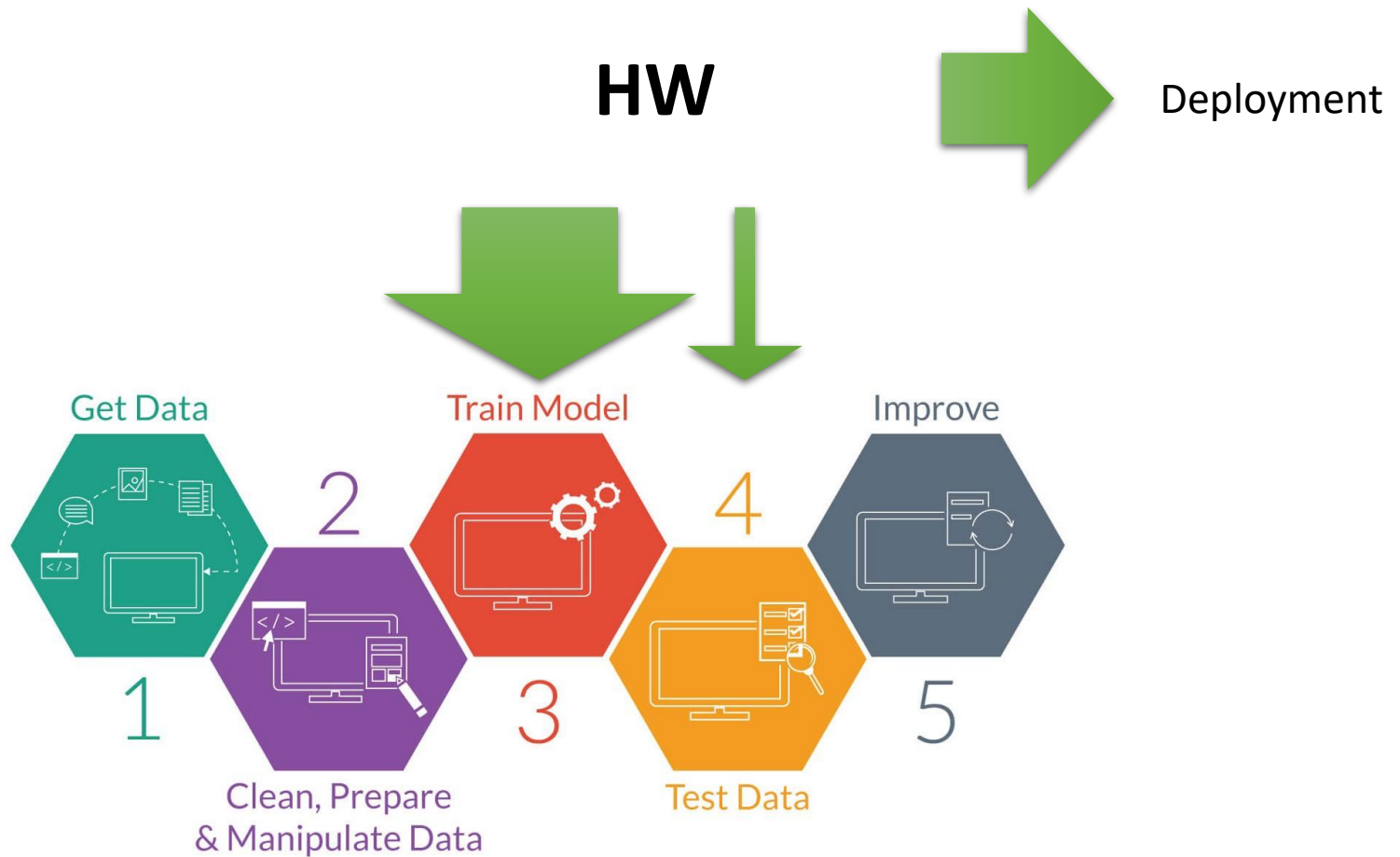
NVIDIA
HGX™ B200



Tensor Processing Unit (TPU)

HW:

When and where is more relevant?



CPU, GPU, TPU, NPU, VPU

CPU

Central Processing Unit



CPU



GPU

GPU

Graphics Processing Unit

TPU

Tensor Processing Unit



NPU

NPU

Neural Processing Unit



TPU

VPU

Vision Processing Unit



VPU

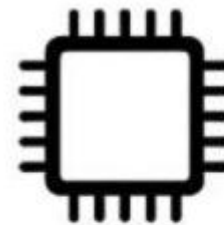
CPU, GPU

CPU (Central Processing Unit)

- General-purpose processor, high flexibility, low parallelism
 - Suitable for prototypes, preprocessing, and system control
 - Not suitable for deep learning training or complex real-time biometric inference

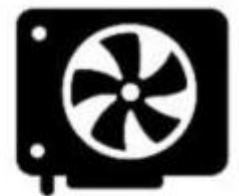
GPU (Graphics Processing Unit)

- Massive parallelism, high memory bandwidth
 - Standard choice for training biometric CNNs
 - Also used for batch inference or real-time inference on desktop/server systems



CPU

General-
purpose &
classic ML



GPU

Parallel
deep
learning

TPU, NPU

TPU (Tensor Processing Unit)

Acceleratore specializzato per operazioni tensoriali

→ **Efficiente per training/inference su modelli standardizzati**

→ Limitata flessibilità, uso tipicamente **cloud-oriented**

NPU (Neural Processing Unit)

Acceleratore per inference neurale a bassa potenza

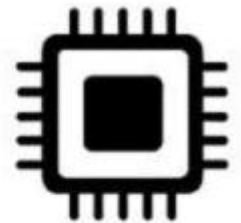
→ **Ideale per biometria embedded e real-time**

→ Ottimizzata per latenza, consumo energetico e privacy on-device



NPU

On-device
edge AI



TPU

Cloud-scale
TensorFlow

VPU

VPU (Vision Processing Unit)

- Specialized accelerator for visual processing and computer vision
 - → Optimized for pre-processing, feature extraction, and inference on video/image data
 - → Ideal for embedded visual biometrics: face, gesture, people counting
 - → Trade-off between GPU and NPU: good energy efficiency and dedicated video pipelines



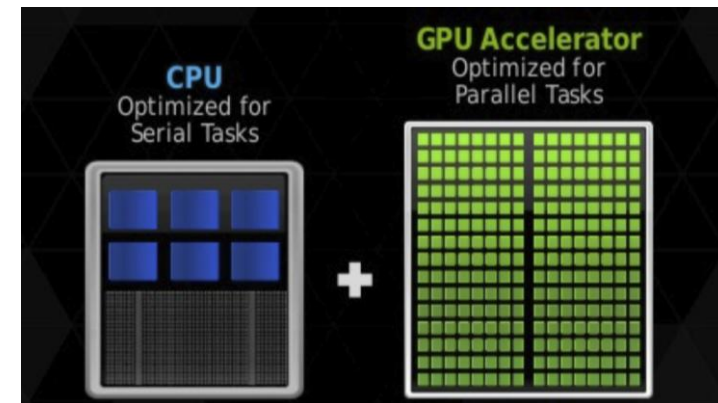
Vision processing unit

GPU vs CPU

In brief....you need a GPU for large projects!

- **Performance (*)**:
about 5x–30x faster (with 10x as a common industry average)
- **Energy efficiency (**)** :
about 3x–10x more efficient
(5x is a realistic common number)

Hardware Setup	Time per Epoch	Total Time (90 Epochs)
2× GTX 580 GPUs (2012)	~1.5 hours	~5–6 days
NVIDIA RTX 2080 Ti	~5 minutes	~7.5 hours
Intel i7 CPU	~2.4 hours	~9 days
NVIDIA DGX-1	~1.3 minutes	~2 hours
Distributed (512 GPUs)	~1 second	~1.5 minutes



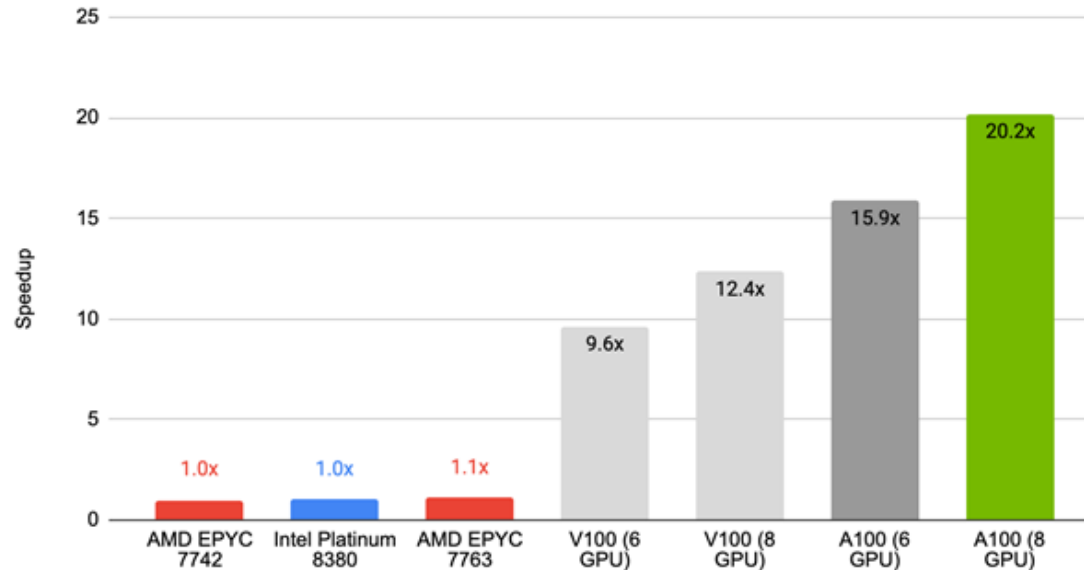
(*) Performance

For highly parallel AI workloads, GPUs can provide large speedups over CPUs, often from a few times faster to more than one order of magnitude. The actual gain depends on model architecture, batch size, precision, memory bandwidth, and data-transfer overhead.

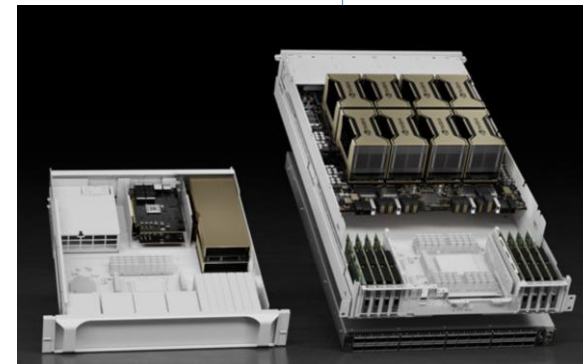
(**) Energy efficiency

GPUs often reduce the energy per trained model or per batch inference because they complete parallel workloads faster. However, absolute power consumption is higher, and efficiency must be measured as performance per watt or energy per inference, not as a fixed universal multiplier.

Some other data



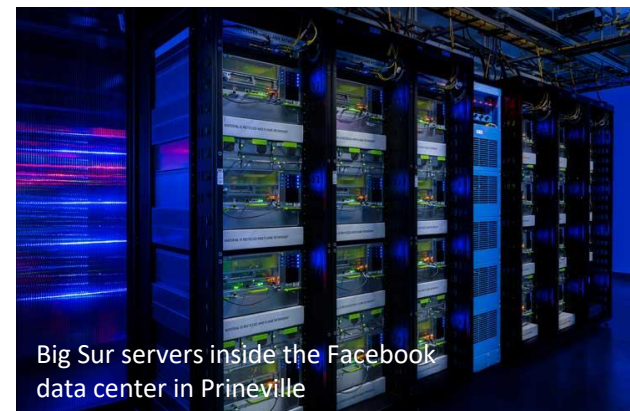
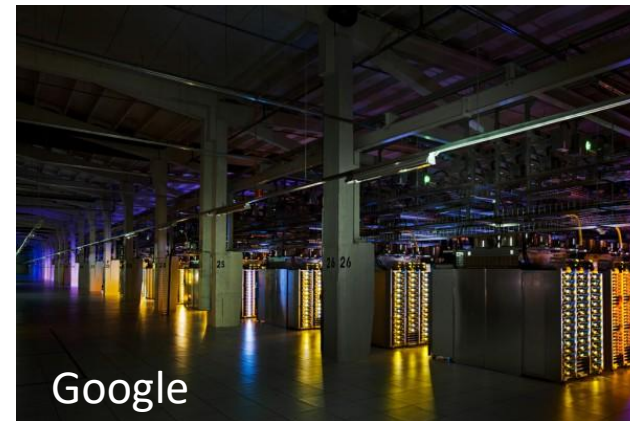
some Keuro



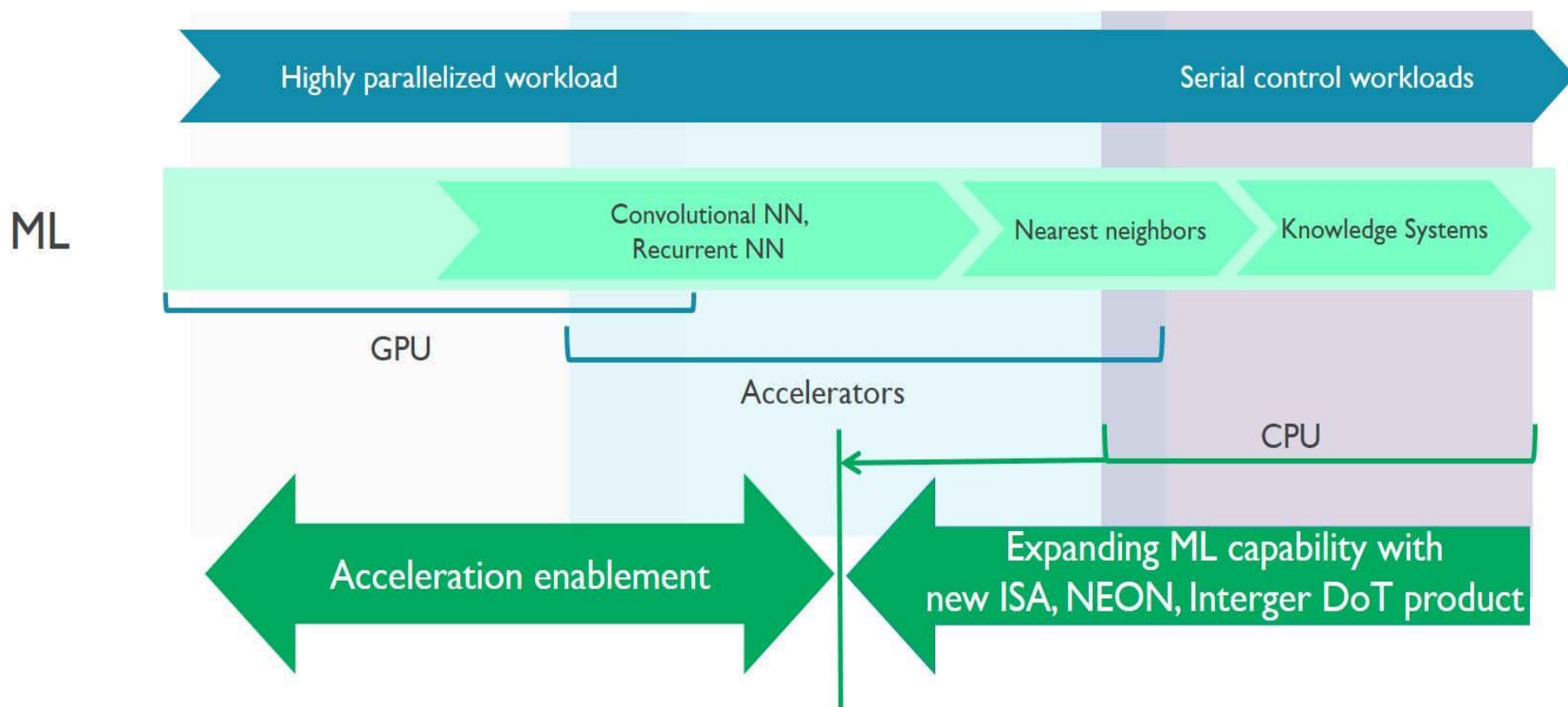
SUPERCOMPUTER
NVIDIA HGX AI
A100 (8GPU)
>100Keuro

ML Server farm...

- Amazon (AWS)
- Google
- Facebook
- Microsoft (AzureML)
- ...



GPUs: not always needed



ML@CPUs

- Machine Learning on CPUs offer **advantages**
 - Ease of **portability** and use-case flexibility
 - Same code, different applications
 - Market availability at different
 - performances
 - **prices**
 - Deployment across a **wide spectrum of devices** from edge to edge compute and cloud servers

Moving ML to the «edge»

Cloud servers



Training +
inference

Edge Compute
Regional servers



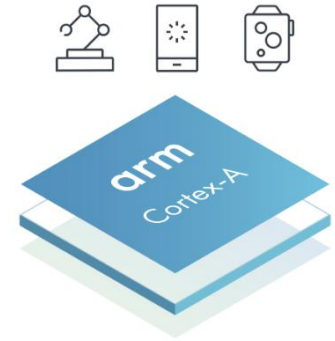
Training +
inference

Edge devices



Sensing, training,
inference & actuation

Example: ML on a ARM proc.

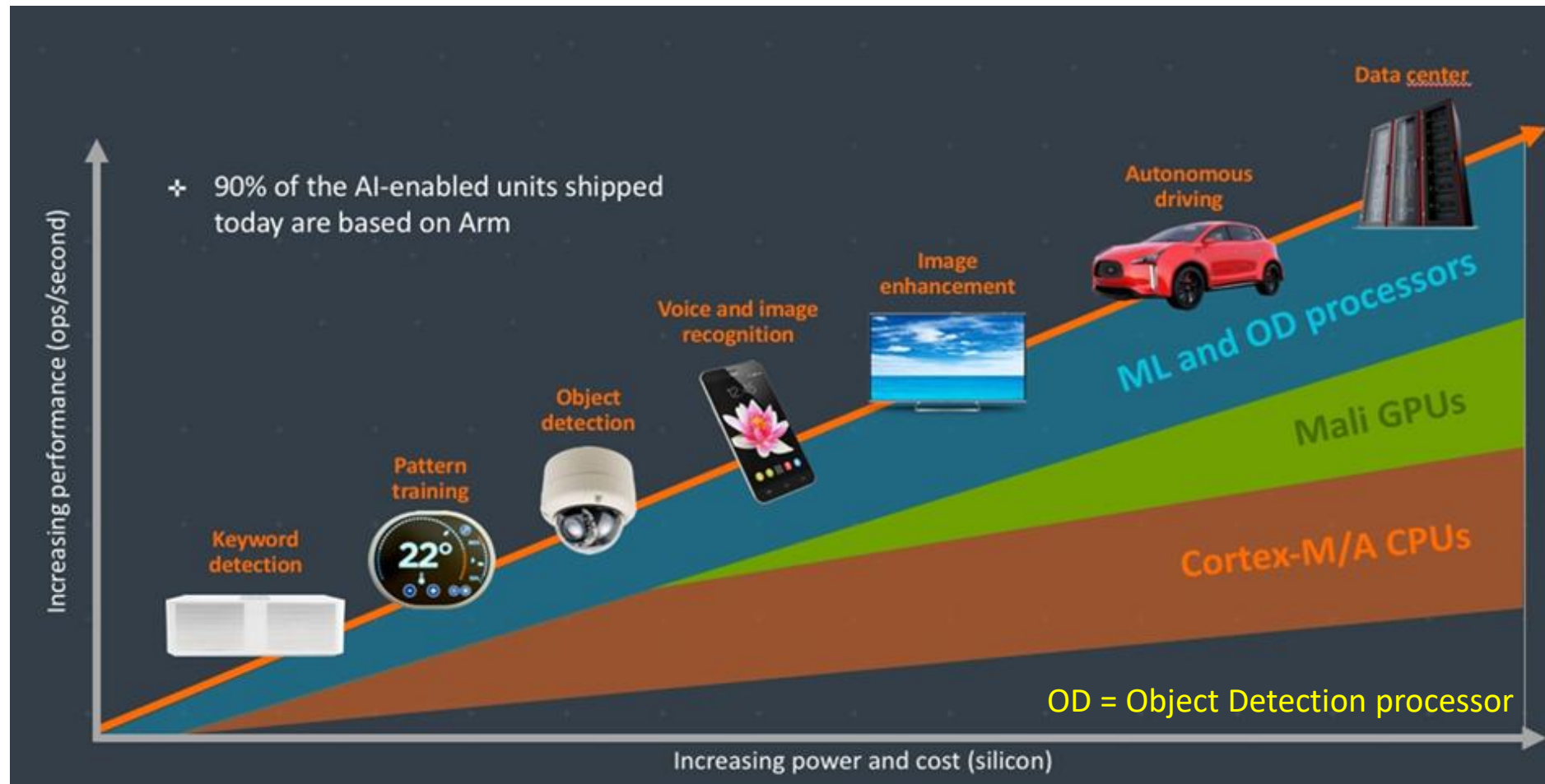


- Arm AI Platform
 - A complete, compute platform including Arm Cortex CPUs and Mali GPUs as well as a new line of highly scalable processors that make the platform versatile enough to scale to any device, from IoT to connected cars and servers.
- Set of functions for
 - ML frameworks like Google's TensorFlow
 - Imaging and vision projects
 - Providing **portable code** that can run across various Arm system configurations.
- The neural networks training considering examples often don't require very high accuracy data, meaning that math calculations can usually be **performed on 16-bit or even 8-bit data**, rather than large 32 or 64-bit entries.
- **The majority of neural network processing uses 8-bit fixed-point matrix multiplication**
 - The Armv8.2-A architecture introduced support for half-precision (FP16) and integer dot products (INT8) floating point SIMD (single instruction multiple data) NEON instructions to accelerate ML NN processing.

HW for ML

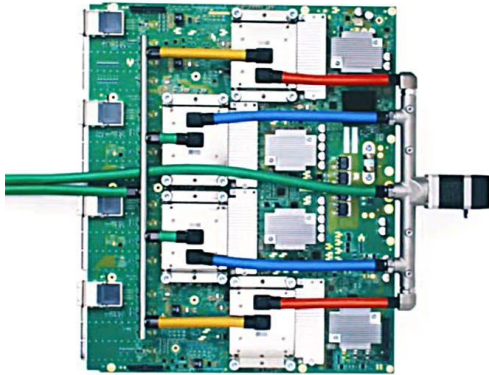
Example: the ARM family

Today, the key point is not only CPU vs GPU. Arm AI systems use heterogeneous computing: CPU for control, GPU for parallel workloads, NPU for low-power neural inference, and optimized libraries to map models efficiently to the available hardware.



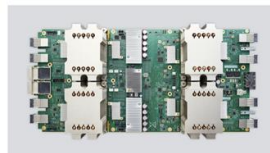
Google TPU

- A tensor processing unit (TPU) is an AI accelerator application-specific integrated circuit (ASIC) developed by Google specifically for neural network machine learning



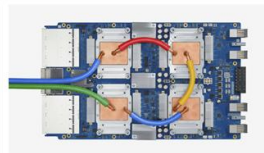
275 Peak TeraFlops

TPU^{v4}



Cloud TPU v2
180 teraFLOPS

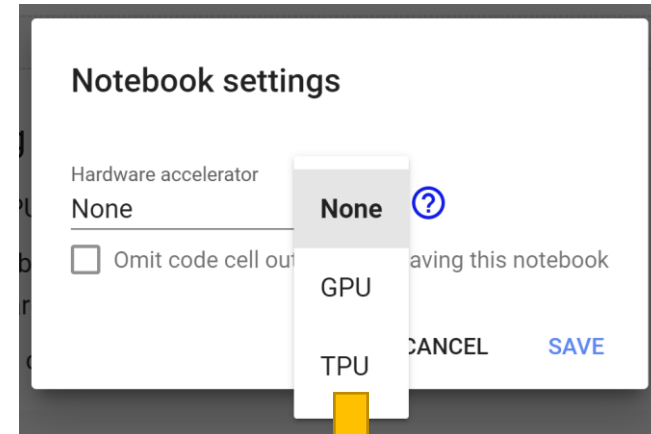
64 GB HBM (High Bandwidth Memory)



Cloud TPU v3
420 teraFLOPS

128 GB HBM

TPUs in Colab

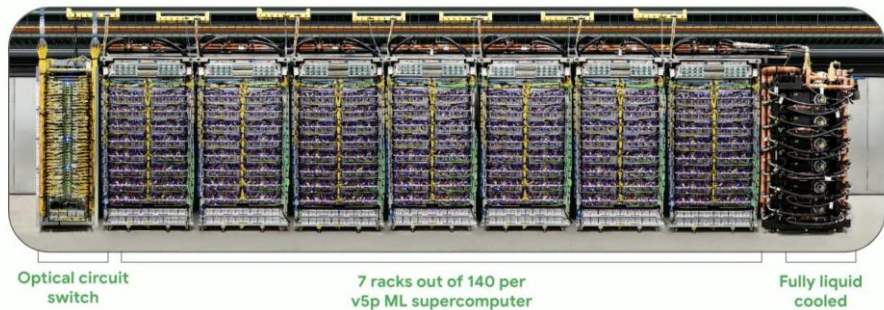


Your CoLab code is (probably) running here

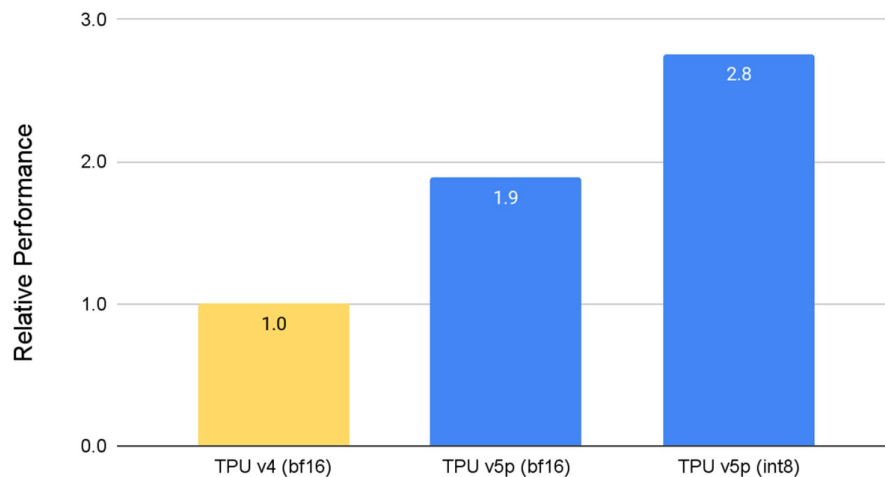
Google TPU v5 and Trillium (the sixth generation)

TPU v5p

Preview



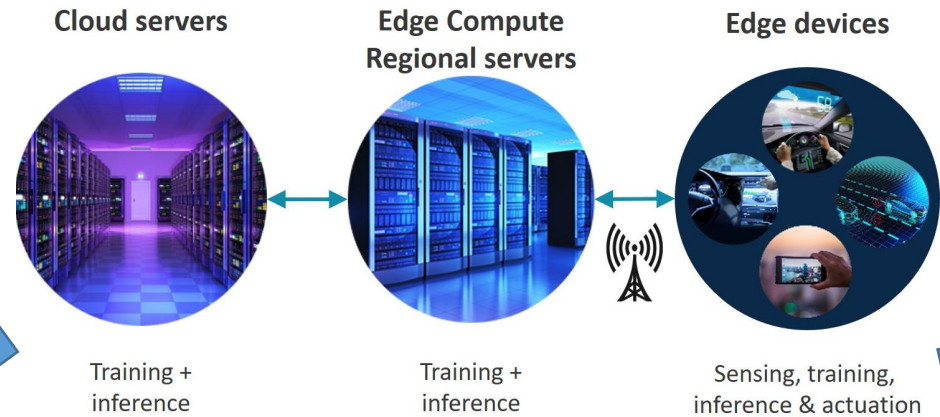
Training Speed (GPT3-175B Training)



Trillium TPUs are a part of Google Cloud's [AI Hypercomputer](#), a groundbreaking supercomputing architecture designed specifically for cutting-edge AI workloads

Our commitment to open-source libraries like JAX, **PyTorch**/XLA, and Keras 3

Moving ML to the «edge»



Exaflop-scale Supercomputers

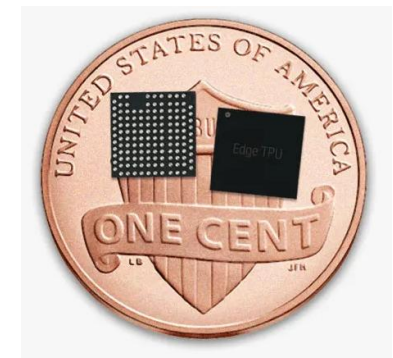


Pichai presenting the Google 9 exaflops supercomputer

TPUs are connected together into supercomputers, called pods containing 4000+ chips

Edge Neural Accelerators

Google Edge TPU



An ASIC designed to run AI at the edge. It delivers high performance in a small physical and power footprint, enabling the deployment of high-accuracy AI at the edge. (Int8 math. It runs a MobileNetv2 @400Frame/s)

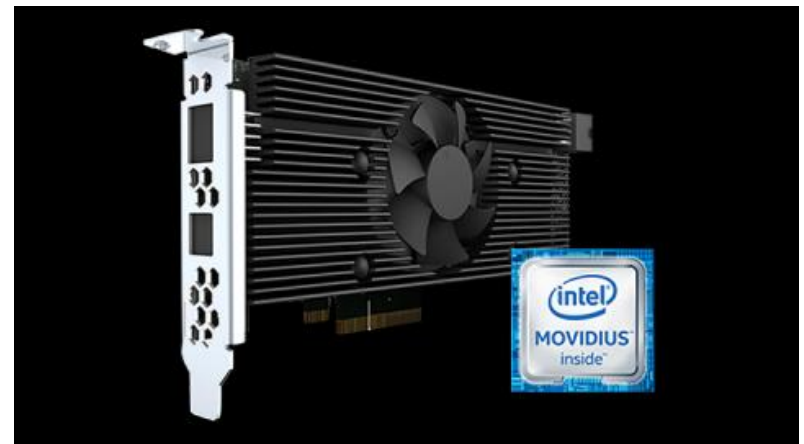
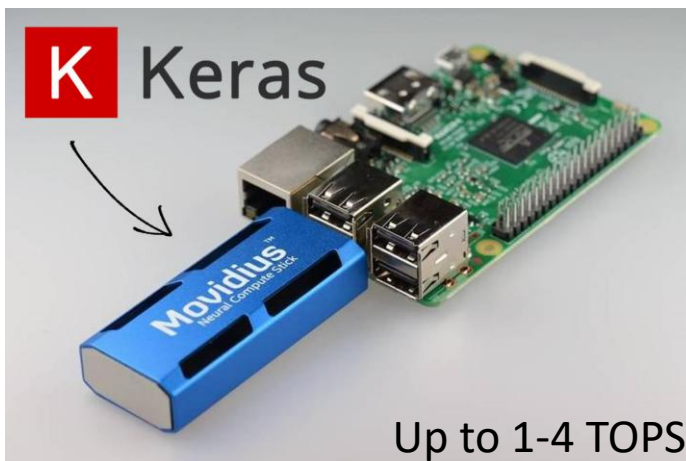
Movidius Neural Compute Stick from Intel

(historical example)

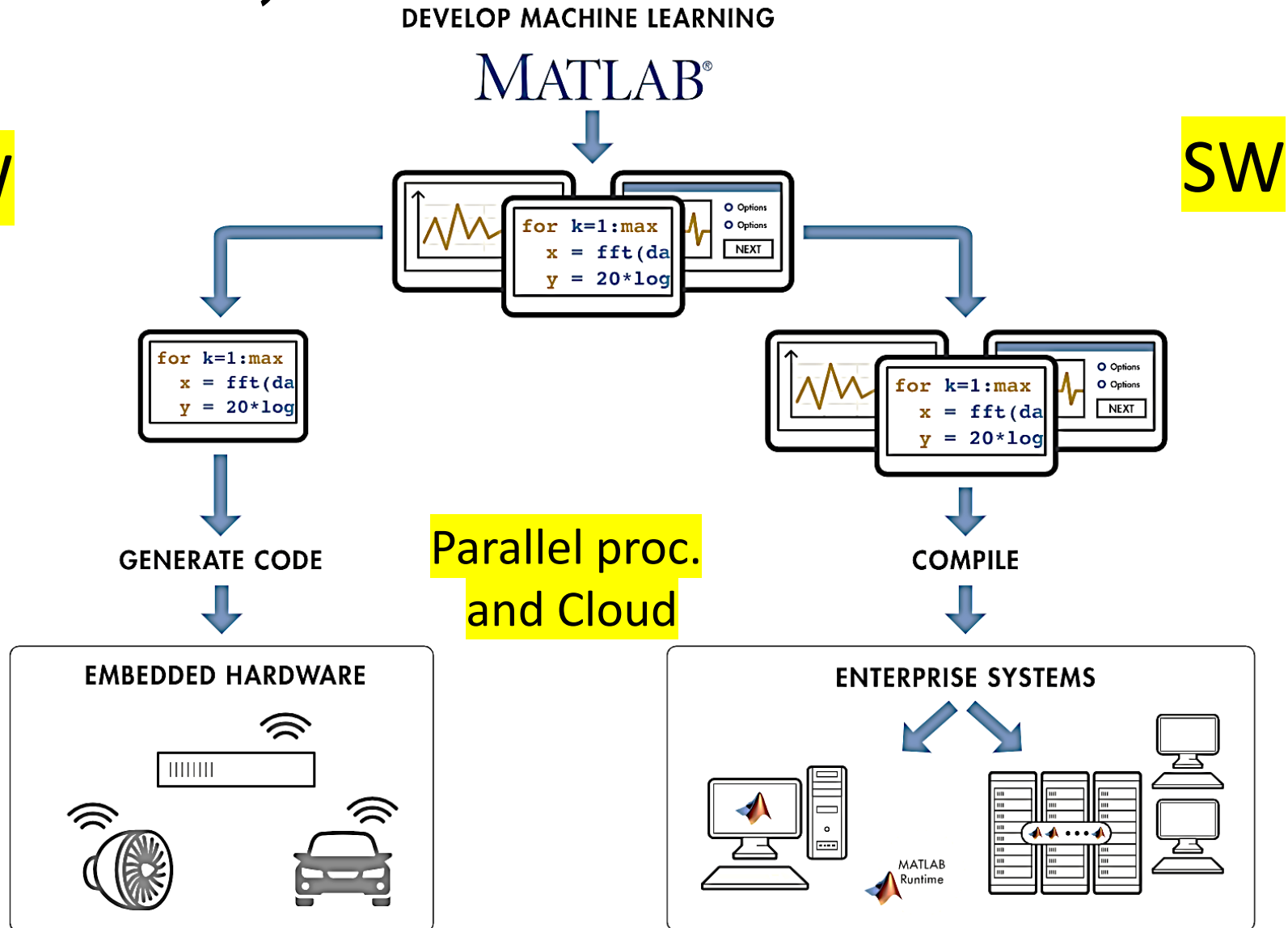
- The Intel Movidius Neural Compute Stick was a compact USB VPU accelerator used to learn and prototype edge-AI inference.
- It is now a legacy platform: Intel discontinued the product, and recent OpenVINO versions no longer support Movidius/Myriad X devices.
- Modern edge AI is usually based on integrated NPUs, embedded GPUs, or dedicated edge accelerators.



Intel® Movidius™
Vision Processing Unit (VPU)



The Matlab environment (HW+SW)

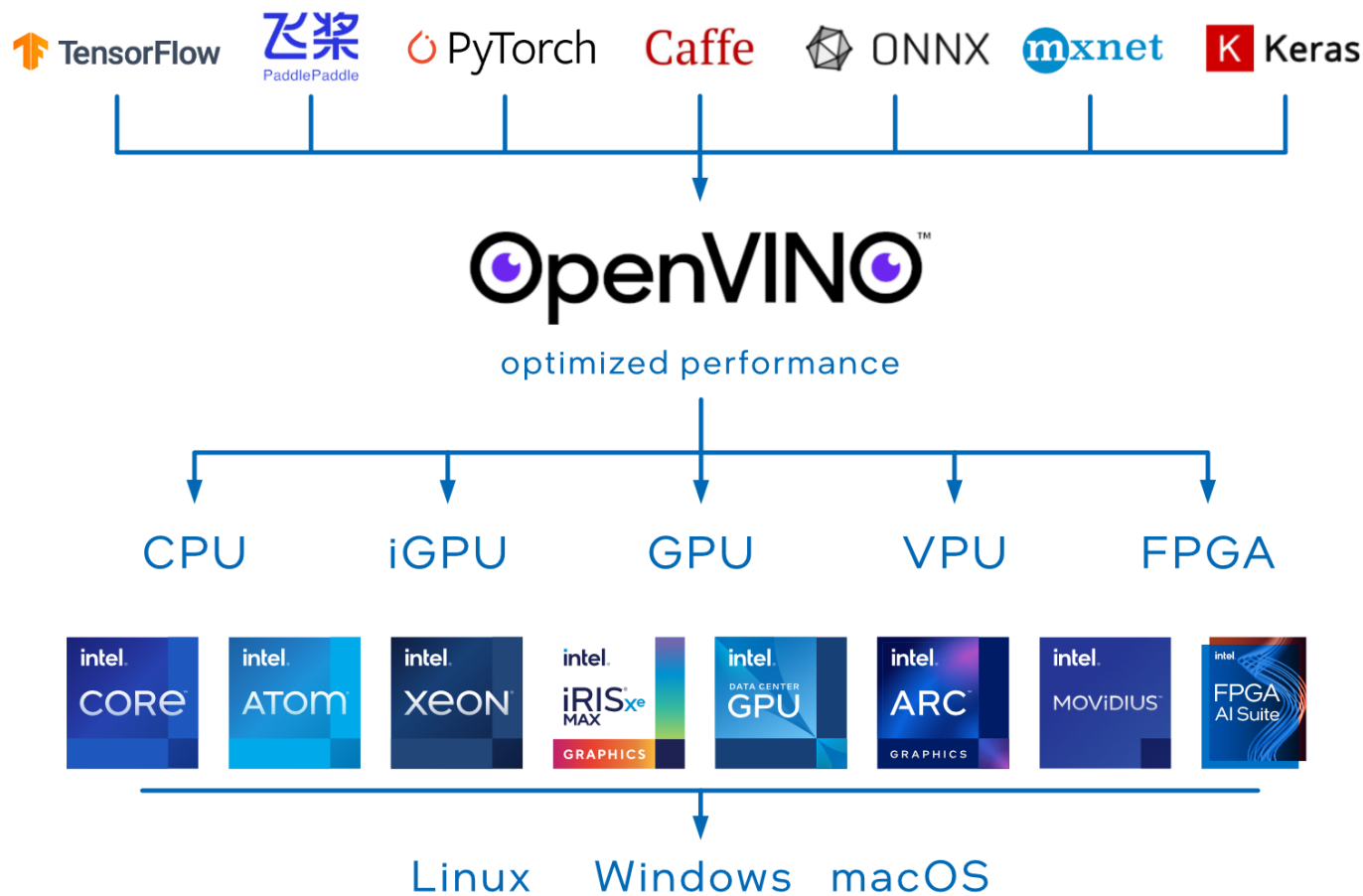




OPENVINO

A powerful optimization platform for models

Intro



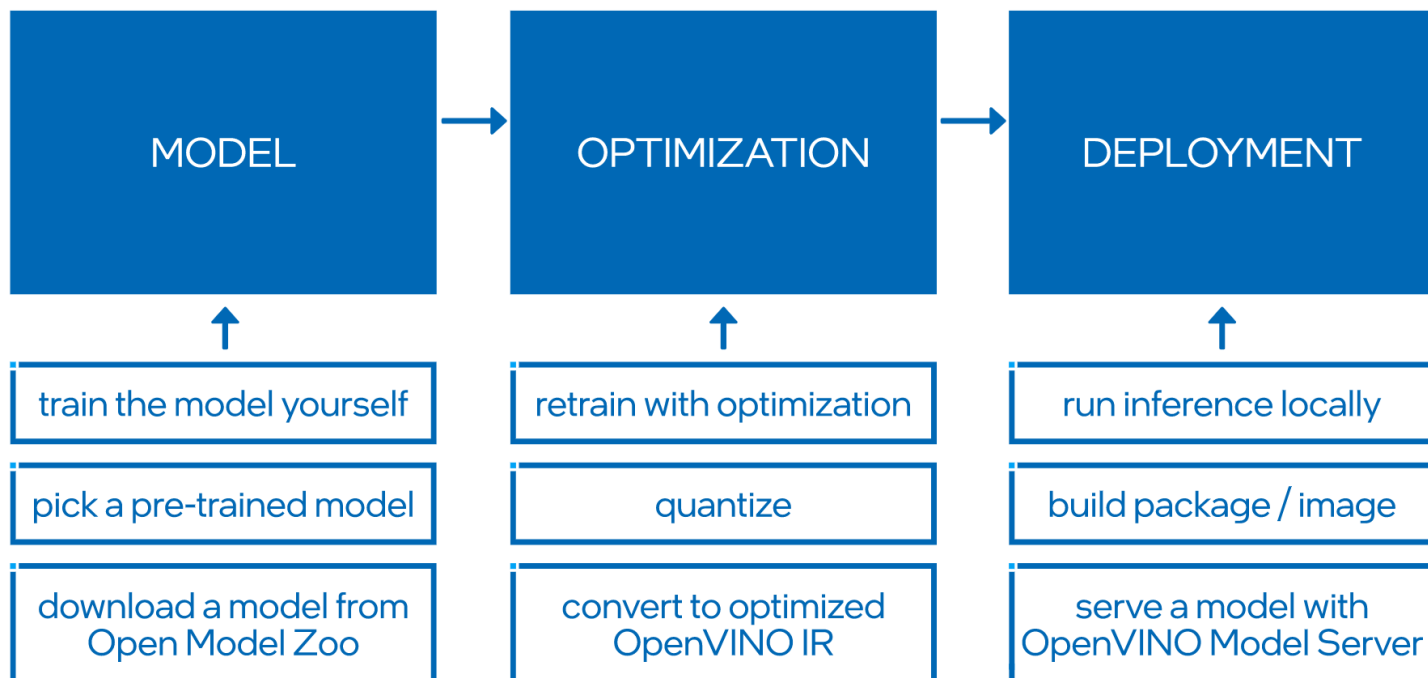
Supported Devices



A typical workflow with OpenVINO

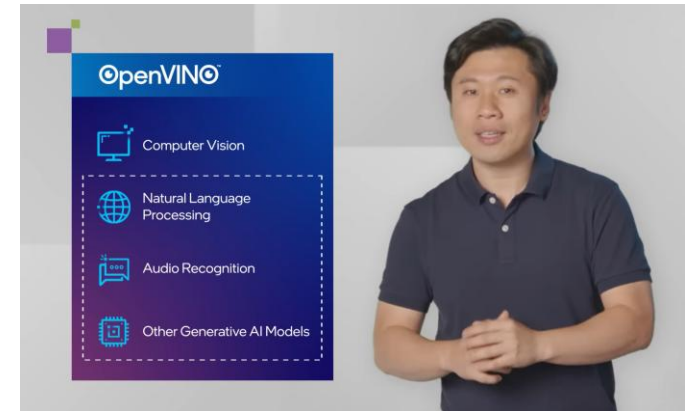
OpenVINO Runtime automatically optimizes deep learning pipelines using aggressive graph fusion, memory reuse, load balancing, and inferencing parallelism across CPU, GPU, VPU

- Post-Training Optimization Tool
- Neural Network Compression Framework

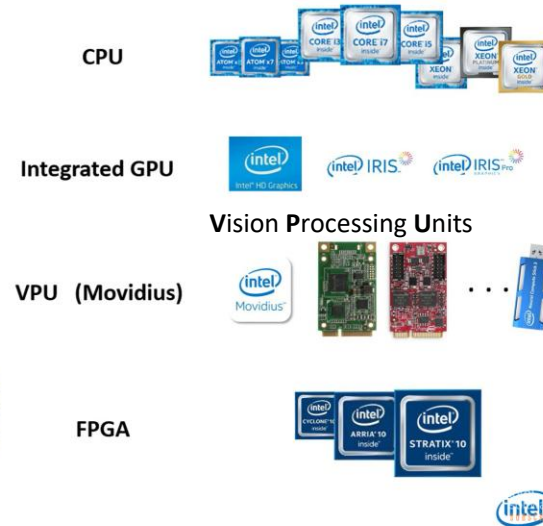


In case you might be interested...

https://www.youtube.com/watch?v=oK7Bum1x_dl



<https://www.youtube.com/watch?v=kY9nZbX1DWM>

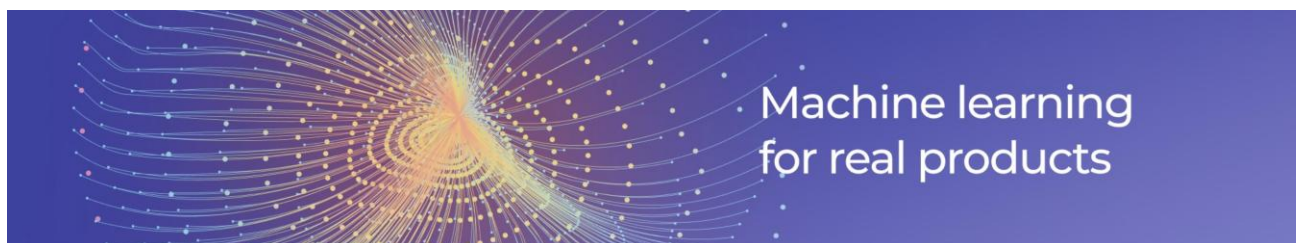


What is OpenVINO? | OpenVINO™ toolkit | Ep. 01 | Intel Software



Edge IMPULSE

A platform for **Edge AI** data collection, training, optimization and deployment



Edge Impulse platform

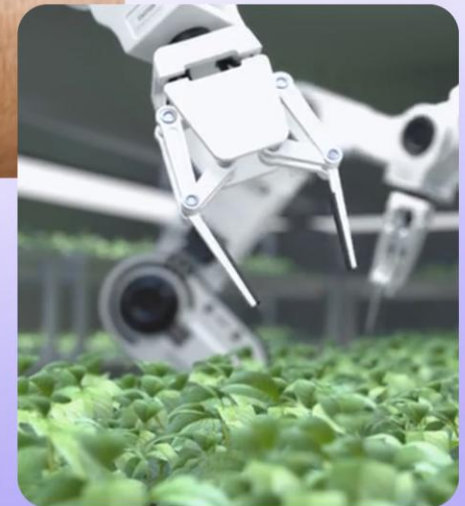
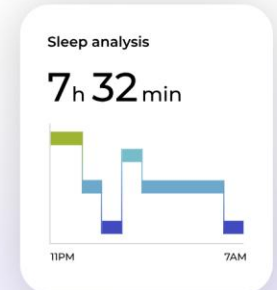
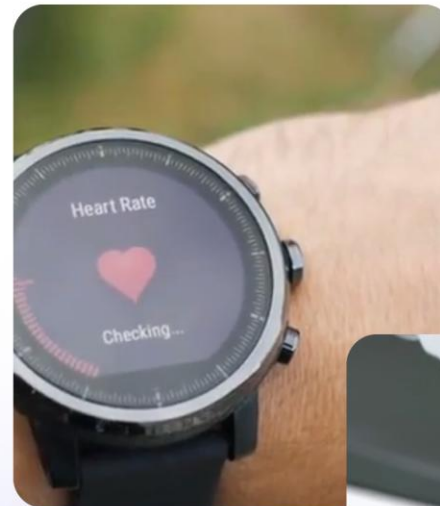
(not in the exam)

Optimize AI for the edge

Edge Impulse is the edge AI platform for enterprise teams building innovative products. Optimize your models and deploy to any edge device with ease. Accelerate product development while minimizing risks with a platform designed to handle real-world sensor data.

[Start today](#)

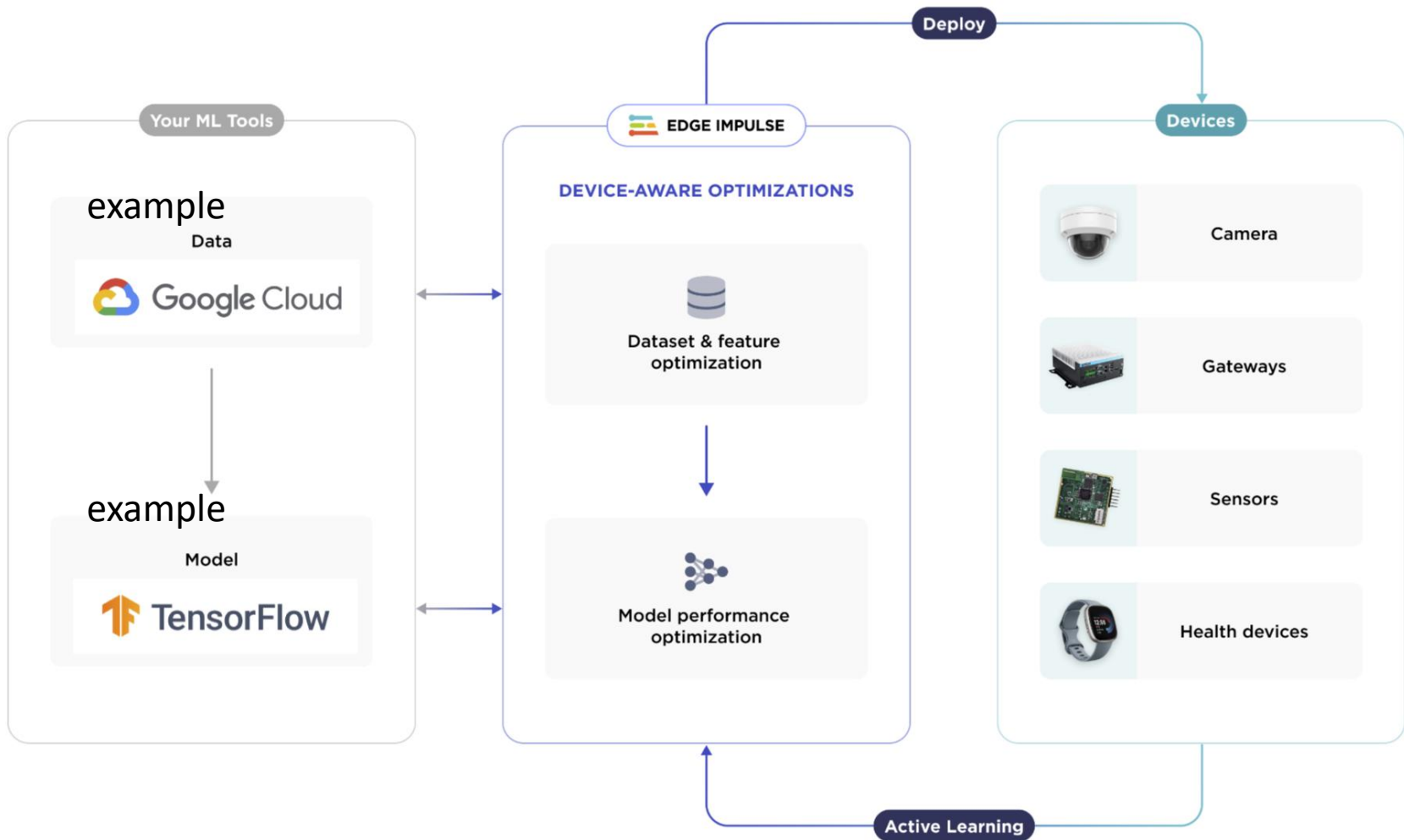
[Schedule a demo](#)



The world's most innovative companies choose Edge Impulse. [Read our case studies >](#)

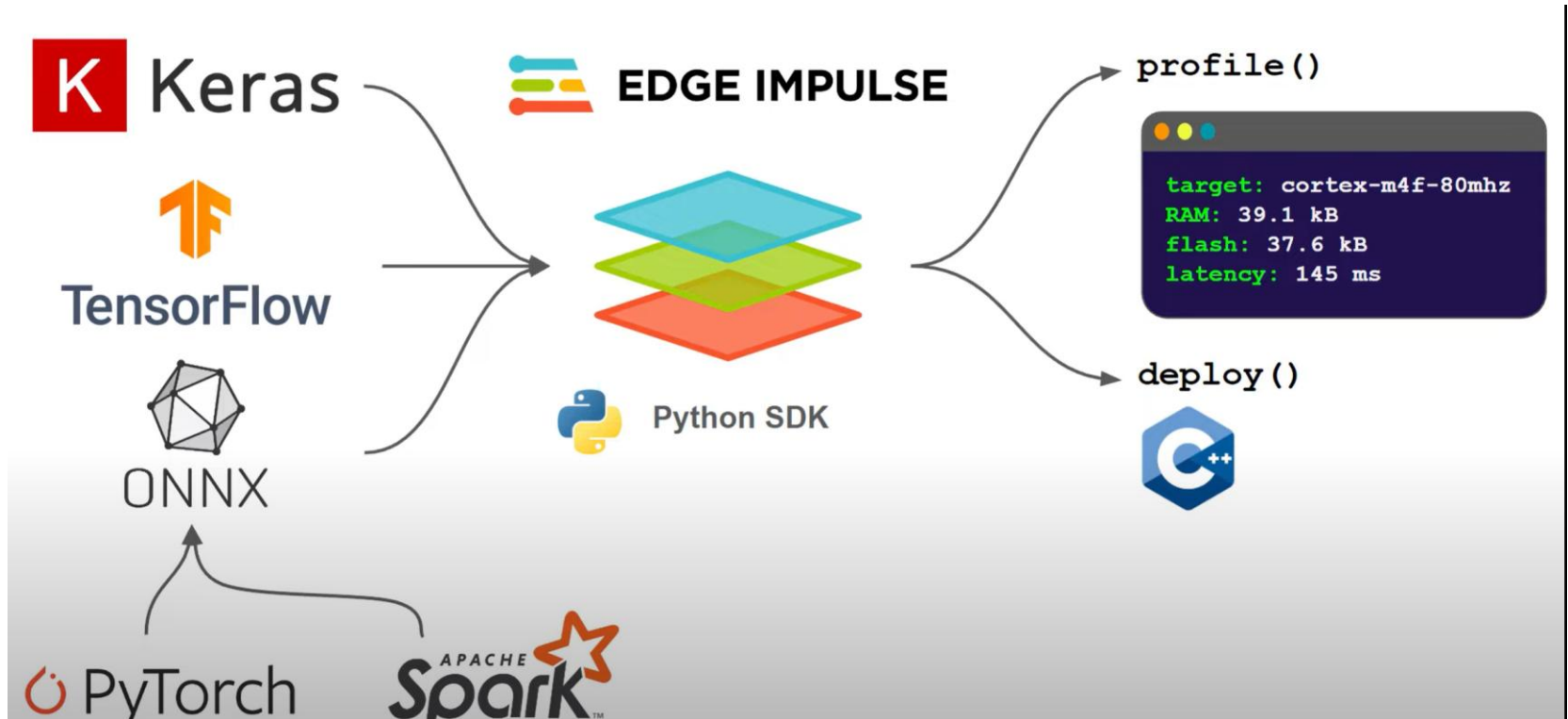
Edge Impulse platform

(not in the exam)



Edge Impulse platform

(not in the exam)



Tutorial: <https://www.youtube.com/watch?v=0Uga3GVXgMw>

Main points



- Intelligent Vision Systems
 - Object detection, Segmentation and Recognition
 - Classical, Deep and Hybrid solutions
 - Use cases
- Machine learning Hardware
 - GPU vs CPU
 - TPU
 - Optimization tools: the OpenVINO example
 - General purpose Python based platform: Edge IMPULSE